

李沐

<https://courses.d2l.ai/zh-v2/>

目录

目录.....	1
一、预备知识.....	5
1、数据操作和数据预处理.....	5
2、线性代数.....	8
3、矩阵计算.....	11
二、线性神经网络.....	14
1、线性回归+基础算法优化.....	14
2、Softmax 回归 + 损失函数 + 图片分类数据集.....	17
2.1 【softmax 回归】.....	17
2.2 损失函数.....	18
三、多层感知机.....	19
1、感知机.....	19
2、【多层感知机】.....	21
3、模型选择.....	23
4、过拟合和欠拟合.....	25
5、权重衰退 weight_decay.....	27
6、丢弃法 dropout.....	29
7、数值稳定性.....	30
8、模型稳定性和激活函数.....	31
四、循环神经网络.....	35
1、序列数据.....	35
2、语言模型.....	37
3、循环神经网络（第一个对于序列模型神经网络） rnn.....	38
4、门控循环单元（gru）.....	40
5、长短期记忆网络（LSTM）.....	43
6、深度循环神经网络.....	45
7、双向循环神经网络.....	46
8、编码器 encode-解码器 decode.....	47
9、序列到序列学习（seq2seq）.....	48
10、束搜索.....	50
五、注意力机制.....	51
1、注意力机制.....	51
2、注意力分数.....	53
3、使用注意力机制的 seq2seq.....	55
4、自注意力和位置编码.....	56
5、Transformer.....	58

6、 BERT	61
7、 优化算法	65

大纲回顾

- 深度学习基础
 - 线性回归
 - Softmax回归
 - 多层感知机
 - 模型选择
 - 过拟合、欠拟合
 - 权重衰退
 - Dropout
 - 数值稳定性
 - 模型初始化和激活函数
- 卷积神经网络
 - LeNet
 - AlexNet
 - VGG
 - NiN
 - GoogleNet
 - ResNet
- 计算机视觉
 - 图片增广
 - 微调
 - R-CNN, SSD, YOLO
 - FCN
 - 样式迁移
- 循环神经网络
 - RNN, GRU, LSTM
 - 深层、双向RNN
 - Seq2Seq
- 注意力机制
 - Seq2seq+attention
 - Transformer
 - BERT
- 性能
 - CPU、GPU、ASIC
 - 使用（多）GPU
 - 分布式
- 应用
 - 房价预测
 - 图片分类
 - 物体检测
 - 语义分割
 - BERT
 - 样式迁移
 - 迁移模型
 - 推理

动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>

还有很多应用、模型没有讲到

- 计算机视觉
 - 人脸识别
 - 体态识别
 - 无人驾驶
 - 图片合成
 - 超分辨率
 - 医学图片
- 自然语言处理
 - 文本分类
 - 文本合成
 - 文本摘要
 - 实体命名识别
- 图神经网络 ✓
- 语音识别
- 时序数据
- 玩游戏 ✓
- 代码生成 ✓
- 音乐
- 推荐系统

个人进阶建议 — 知识

- 可以看朋友圈分享，但更要主动获取信息、建立知识库
- 学会读论文
 - 经典论文需要读懂每一句话
 - 结合代码来了解细节（例如 <https://paperswithcode.com>）
 - 可以看openreview上的评（吐）论（槽）
- 对读过的论文做整理
 - <https://www.bilibili.com/video/BV1nA4115>

个人进阶建议 — 实践

- 可以参加竞赛，但注意竞赛跟研究和工业界落地都不一样
 - 竞赛：调最好的参、模型融合
 - 研究：新的模型、调还不错的参
 - 工业界：将应用表达成机器学习问题、收集数据
- 多研究开源代码，跟开发者多交流，积极贡献
 - 你可以从修文档开始



【目标】

- 介绍深度学习经典和最新模型
 - LeNet（最老的），ResNet（计算机视觉），LSTM（时序模型），BERT（最近流行），...
- 机器学习基础
 - 损失函数、目标函数、过拟合、优化
- 实践
 - 使用 Pytorch 实现介绍的知识点
 - 在真实数据上体验算法效果

【内容】

- 深度学习基础—线性神经网络，多层感知机
- 卷积神经网络— LeNet, AlexNet, VGG, Inception, ResNet
- 循环神经网络— RNN, GRU, LSTM, seq2seq
- 注意力机制— Attention, Transformer
- 优化算法— SGD, Momentum, Adam
- 高性能计算— 并行，多 GPU，分布式
- 计算机视觉— 目标检测，语义分割
- 自然语言处理— 词嵌入，BERT

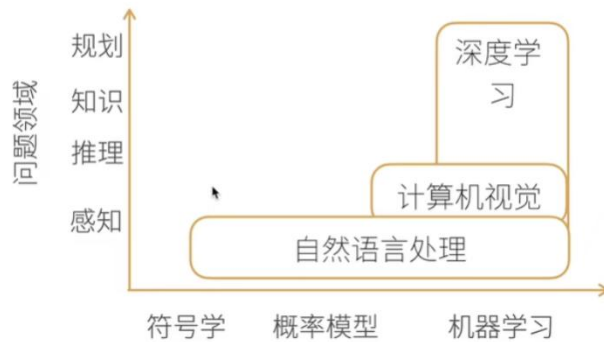
你将学到什么

- What
 - 深度学习里有哪些技术
- How
 - 如何实现和调参
- Why
 - 背后的原因（直觉、数学）



深度学习介绍

AI 地图



自然语言处理：机器翻译

计算机视觉：在图片里往前推（用概率模型、机器学习）

深度学习：机器学习的一种

深度学习应用突破：图片分类、物体检测（想知道图片里的物体在什么地方）和分割（每一个像素属于飞机还是人）、样式迁移、人脸合成、文字生成图片、文字生成模型、无人驾驶

案例研究：广告点击（预测点击率）

触发广告-点击率预估-排序（点击率*竞价）

预测与训练



完整的故事



领域专家：对应用有深的了解，用户的行为会怎样，模型训练进行拟合，更关心的是机器学习的模型会对产品带来什么样的影响（提需求的人-甲方）

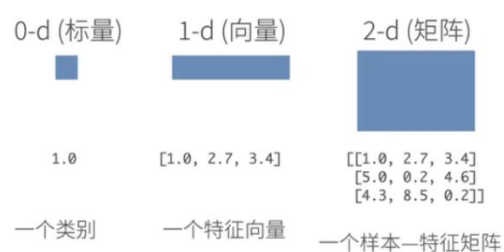
数据科学家：拿到的是数据，把原始的数据变成机器学习能理解的数据，对模型进行训练然后上线

可解释性-为什么有效和可解释性是两件事情，可解释性是人是不是能理解这个模型，他什么时候会工作以及在什么时候会出现偏差

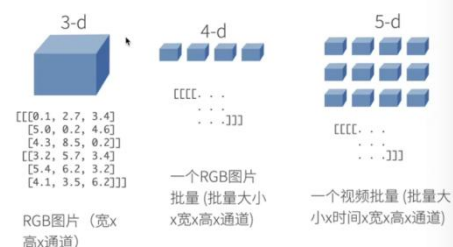
一、预备知识

1、数据操作和数据预处理

机器学习用的最多的是N维数组



N维数组样例（续）

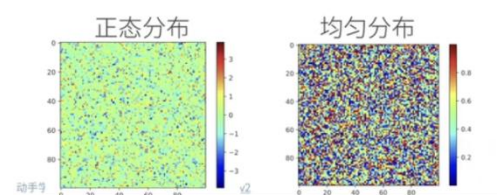


4 维-n 个 3 维数组放在一起

5 维-视频的批量-多加了时间维度

创建数组

- 创建数组需要
 - 形状：例如 3 x 4 矩阵
 - 每个元素的数据类型：例如32位浮点数
 - 每个元素的值，例如全是0，或者随机数



访问元素



一个元素: [1, 2]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

一行: [1, :]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

一列: [1, :]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

子区域: [1:3, 1:]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

子区域: [::3, ::2]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

行, 列

子区域: 开区间

子区域: 跳着访问-每三行一跳, 每两列一跳

张量表示一个数值组成的数组

```
In [2]: x = torch.arange(12)
x
Out[2]: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

● 数据预处理

➤ 处理缺失值

注意, “NaN”项代表缺失值。为了处理缺失的数据, 典型的方法包括插值法和删除法, 其中插值法用一个替代值弥补缺失值, 而删除法则直接忽略缺失值。在这里, 我们将考虑插值法。

```
import os
```

```
os.makedirs(os.path.join('.', 'data'), exist_ok=True)
```

```
data_file = os.path.join('.', 'data', 'house_tiny.csv')
```

```
with open(data_file, 'w') as f:
```

```
    f.write('NumRooms,Alley,Price\n') # 列名
```

```
    f.write('NA,Pave,127500\n') # 每行表示一个数据样本
```

```
    f.write('2,NA,106000\n')
```

```
f.write('4,NA,178100\n')
f.write('NA,NA,140000\n')
```

```
import pandas as pd
data = pd.read_csv(data_file)
print(data)
```

	NumRooms	Alley	Price
0	NaN	Pave	127500
1	2.0	NaN	106000
2	4.0	NaN	178100
3	NaN	NaN	140000

通过位置索引 `iloc`，我们将 `data` 分成 `inputs` 和 `outputs`，其中前者为 `data` 的前两列，而后者为 `data` 的最后一列。对于 `inputs` 中缺少的数值，我们用同一列的均值替换“NaN”项。

```
inputs, outputs = data.iloc[:, 0:2], data.iloc[:, 2]
inputs = inputs.fillna(inputs.mean())
print(inputs)
```

	NumRooms	Alley
0	3.0	Pave
1	2.0	NaN
2	4.0	NaN
3	3.0	NaN

对于 `inputs` 中的类别值或离散值，我们将“NaN”视为一个类别。由于“巷子类型”（“Alley”）列只接受两种类型的类别值“Pave”和“NaN”，`pandas` 可以自动将此列转换为两列“Alley_Pave”和“Alley_nan”。巷子类型为“Pave”的行会将“Alley_Pave”的值设置为 1，“Alley_nan”的值设置为 0。缺少巷子类型的行会将“Alley_Pave”和“Alley_nan”分别设置为 0 和 1。

```
inputs = pd.get_dummies(inputs, dummy_na=True)
print(inputs)
```

	NumRooms	Alley_Pave	Alley_nan
0	3.0	1	0
1	2.0	0	1
2	4.0	0	1
3	3.0	0	1

➤ 转换为张量格式

现在 `inputs` 和 `outputs` 中的所有条目都是数值类型，它们可以转换为张量格式。当数据采用张量格式后，可以通过在 2.1 节中引入的那些张量函数来进一步操作。

```
from mxnet import np
```

```
X, y = np.array(inputs.to_numpy(dtype=float)), np.array(outputs.to_numpy(dtype=float))
X, y
```

```
(array([[3., 1., 0.],
       [2., 0., 1.],
       [4., 0., 1.],
       [3., 0., 1.]], dtype=float64),
 array([127500., 106000., 178100., 140000.], dtype=float64))
```

2、线性代数

标量：一行/一列值

向量：通过一维张量表示向量。一般来说，张量可以具有任意长度，取决于机器的内存限制。

标量

• 简单操作

$$c = a + b$$

$$c = a \cdot b$$

$$c = \sin a$$

• 长度

$$|a| = \begin{cases} a & \text{if } a > 0 \\ -a & \text{otherwise} \end{cases}$$

$$|a + b| \leq |a| + |b|$$

$$|a \cdot b| = |a| \cdot |b|$$

向量

• 简单操作

$$c = a + b \quad \text{where } c_i = a_i + b_i$$

$$c = \alpha \cdot b \quad \text{where } c_i = \alpha b_i$$

$$c = \sin a \quad \text{where } c_i = \sin a_i$$

• 长度

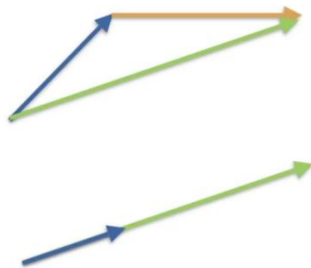
$$\|a\|_2 = \left[\sum_{i=1}^m a_i^2 \right]^{\frac{1}{2}}$$

$$\|a\| \geq 0 \text{ for all } a$$

$$\|a + b\| \leq \|a\| + \|b\|$$

$$\|a \cdot b\| = |a| \cdot \|b\|$$

向量



数学家的 'parallel for all do'

向量

$c = a + b$ • 点乘

$$a^T b = \sum_i a_i b_i$$

$c = \alpha \cdot b$ • 正交

$$a^T b = \sum_i a_i b_i = 0$$



矩阵

• 简单操作

$$C = A + B \quad \text{where } C_{ij} = A_{ij} + B_{ij}$$

$$C = \alpha \cdot B \quad \text{where } C_{ij} = \alpha B_{ij}$$

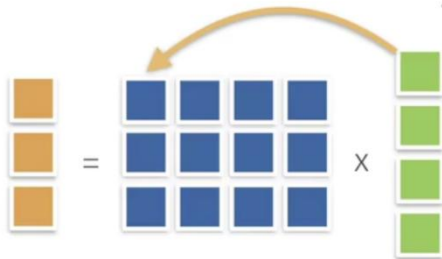
$$C = \sin A \quad \text{where } C_{ij} = \sin A_{ij}$$



矩阵

- 乘法（矩阵乘以向量）

$$c = Ab \text{ where } c_i = \sum_j A_{ij} b_j$$



矩阵乘法从直观上来说是个扭曲空间



矩阵

- 范数

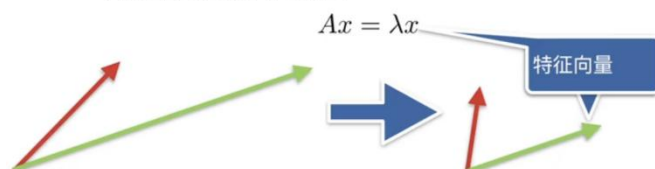
$$c = A \cdot b \text{ hence } \|c\| \leq \|A\| \cdot \|b\|$$

- 取决于如何衡量 b 和 c 的长度
- 常见范数
 - 矩阵范数：最小的满足的上面公式的值
 - Frobenius 范数

$$\|A\|_{\text{Frob}} = \left[\sum_{ij} A_{ij}^2 \right]^{\frac{1}{2}}$$

动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>

- 特征向量和特征值
 - 不被矩阵改变方向的向量



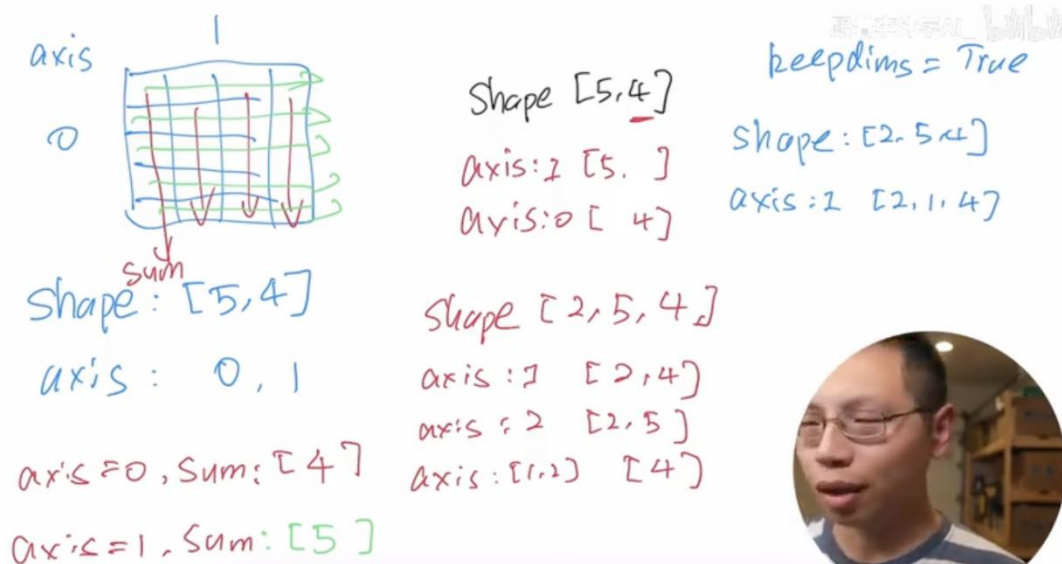
- 对称矩阵总是可以找到特征向量

行是最后一位——4
列是倒数第二位——3
维度是2

```
In [10]: x = torch.arange(24).reshape(2, 3, 4)
x
```

```
Out[10]: tensor([[[ 0,  1,  2,  3],
                  [ 4,  5,  6,  7],
                  [ 8,  9, 10, 11]],
                [[12, 13, 14, 15],
                 [16, 17, 18, 19],
                 [20, 21, 22, 23]]])
```

按某一维度求和，不想把某个维度丢掉——keepdims=True
——这样可以通过广播机制 A/sum_A



计算总和或均值时保持轴数不变

DEEPLARNING

```
In [21]: sum_A = A.sum(axis=1, keepdims=True)
          sum_A

Out[21]: tensor([[ 6.],
                  [22.],
                  [38.],
                  [54.],
                  [70.]])
```

通过广播将 A 除以 sum_A

```
In [22]: A / sum_A

Out[22]: tensor([[0.0000, 0.1667, 0.3333, 0.5000],
                  [0.1818, 0.2273, 0.2727, 0.3182],
                  [0.2105, 0.2368, 0.2632, 0.2895],
                  [0.2222, 0.2407, 0.2593, 0.2778],
                  [0.2286, 0.2429, 0.2571, 0.2714]])
```

L_2 范数是向量元素平方和的平方根:

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

```
In [28]: u = torch.tensor([3.0, -4.0])
          torch.norm(u)

Out[28]: tensor(5.)
```

3、矩阵计算

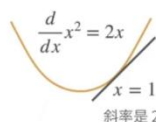
——讲矩阵怎么求导数，所有优化模型的求解都是通过求导数来进行的

——理解导数的形状和 input 形状的关系

y	a	x^n	$\exp(x)$	$\log(x)$	$\sin(x)$
$\frac{dy}{dx}$	0	nx^{n-1}	$\exp(x)$	$\frac{1}{x}$	$\cos(x)$
a 不是 x 的函数					

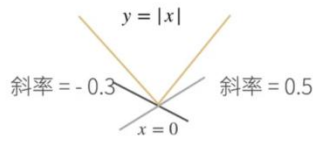
y	$u+v$	uv	$y=f(u), u=g(x)$
$\frac{dy}{dx}$	$\frac{du}{dx} + \frac{dv}{dx}$	$\frac{du}{dx}v + \frac{dv}{dx}u$	$\frac{dy}{du} \frac{du}{dx}$

导数是切线的斜率



亚导数

- 将导数拓展到不可微的函数



$$\frac{\partial |x|}{\partial x} = \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{if } x < 0 \\ a & \text{if } x = 0, a \in [-1, 1] \end{cases}$$

另一个例子

$$\frac{\partial}{\partial x} \max(x, 0) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \\ a & \text{if } x = 0, a \in [0, 1] \end{cases}$$



向量链式法则

- 标量链式法则

$$y = f(u), u = g(x) \quad \frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial x}$$

- 拓展到向量

$$\begin{matrix} \frac{\partial y}{\partial \mathbf{x}} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial \mathbf{x}} & \frac{\partial y}{\partial \mathbf{x}} = \frac{\partial y}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}} & \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}} \\ (1, n) \quad (1,) \quad (1, n) & (1, n) \quad (1, k) \quad (k, n) & (m, n) \quad (m, k) \quad (k, n) \end{matrix}$$

假设

$$\mathbf{x}, \mathbf{w} \in \mathbb{R}^n, y \in \mathbb{R}$$

$$z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$$

计算

$$\begin{aligned} \frac{\partial z}{\partial \mathbf{w}} &= \frac{\partial z}{\partial b} \frac{\partial b}{\partial a} \frac{\partial a}{\partial \mathbf{w}} \\ &= \frac{\partial b^2}{\partial b} \frac{\partial a - y}{\partial a} \frac{\partial \langle \mathbf{x}, \mathbf{w} \rangle}{\partial \mathbf{w}} \\ &= 2b \cdot 1 \cdot \mathbf{x}^T \\ &= 2(\langle \mathbf{x}, \mathbf{w} \rangle - y) \mathbf{x}^T \\ &= 2z \mathbf{x}^T \end{aligned}$$

分解

$$\begin{aligned} a &= \langle \mathbf{x}, \mathbf{w} \rangle \\ b &= a - y \\ z &= b^2 \end{aligned}$$

动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>

自动求导

- 自动求导计算一个函数在指定值上的导数
- 它有别于
 - 符号求导

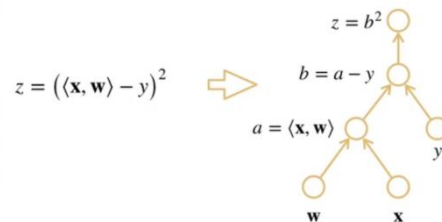
$$\begin{aligned} \text{In[1]} &:= \text{D}[4x^3 + x^2 + 3, x] \\ \text{Out[1]} &= 2x + 12x^2 \end{aligned}$$

- 数值求导

$$\frac{\partial f(x)}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

计算图

- 将代码分解成操作子
- 将计算表示成一个无环图



计算图等价于用链式法则的求解过程
每个圈表示一个操作/输入

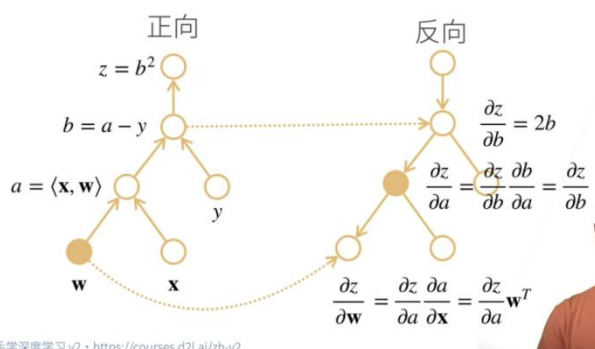
自动求导的两种模式

- 链式法则: $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \dots \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x}$
- 正向累积 $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n} \left(\frac{\partial u_n}{\partial u_{n-1}} \left(\dots \left(\frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x} \right) \right) \right)$
- 反向累积、又称反向传递

$$\frac{\partial y}{\partial x} = \left(\left(\left(\frac{\partial y}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \right) \dots \right) \frac{\partial u_2}{\partial u_1} \right) \frac{\partial u_1}{\partial x}$$

反向累积

$$z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$$



反向累积总结

•构造计算图

- 前向：执行图，存储中间结果
- 反向：从相反方向执行图
 - 去除不需要的枝

复杂度

- 计算复杂度: $O(n)$, n 是操作子个数
 - 通常正向和反向的代价类似
- 内存复杂度: $O(n)$, 因为需要存储正向的所有中间结果
 - 深度神经网络特别耗 gpu 资源的祸源, 求梯度的时候需要把结果存下来
- 跟正向累积对比:
 - $O(n)$ 计算复杂度用来计算一个变量的梯度
 - $O(1)$ 内存复杂度——不需要存任何结果

自动求导实现

在我们计算 y 关于 x 的梯度之前，我们需要一个地方来存储梯度。

```
In [2]: x.requires_grad_(True) # 等价于 `x = torch.arange(4.0, requires_grad=True)`  
x.grad # 默认值是None
```

现在让我们计算 y 。

```
In [3]: y = 2 * torch.dot(x, x)  
y  
Out[3]: tensor(28., grad_fn=<MulBackward0>)
```

通过调用反向传播函数来自动计算 y 关于 x 每个分量的梯度

```
In [4]: y.backward()  
x.grad  
Out[4]: tensor([ 0.,  4.,  8., 12.])  
In [5]: x.grad == 4 * x  
Out[5]: tensor([ True,  True,  True,  True])
```

二、线性神经网络

1、线性回归+基础算法优化

【房价预测】

一个简化模型

- 假设 1: 影响房价的关键因素是卧室个数，卫生间个数，和居住面积，记为 x_1, x_2, x_3
- 假设 2: 成交价是关键因素的加权和

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

权重和偏差的实际值在后面决定



线性模型

- 给定 n 维输入 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$
- 线性模型有一个 n 维权重和一个标量偏差

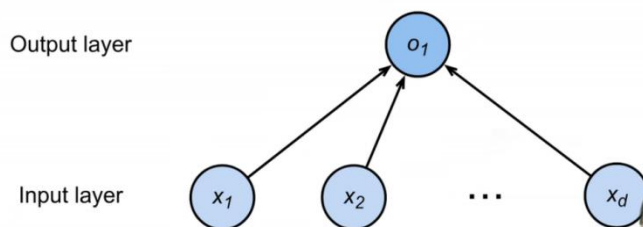
$$\mathbf{w} = [w_1, w_2, \dots, w_n]^T, \quad b$$

- 输出是输入的加权和

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

向量版本: $y = \langle \mathbf{w}, \mathbf{x} \rangle + b$

线性模型可以看作是单层的神经网络



衡量预估质量

- 比较真实值和预估值，例如房屋售价和估价
- 假设 y 是真实值， $\hat{y}$ 是估计值，我们可以比较

$$\ell(y, \hat{y}) = \frac{1}{2} (y - \hat{y})^2$$

这个叫做平方损失

训练数据

- 收集一些数据点来决定参数值（权重和偏差），例如过去6个月卖的房子
- 这被称之为训练数据
- 通常越多越好
- 假设我们有 n 个样本，记

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^T \quad \mathbf{y} = [y_1, y_2, \dots, y_n]^T$$

参数学习

- 训练损失

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b) = \frac{1}{2n} \sum_{i=1}^n (y_i - \langle \mathbf{x}_i, \mathbf{w} \rangle - b)^2 = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w} - b\|^2$$

- 最小化损失来学习参数

$$\mathbf{w}^*, b^* = \arg \min_{\mathbf{w}, b} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b)$$

显示解

- 将偏差加入权重 $\mathbf{X} \leftarrow [\mathbf{X}, \mathbf{1}] \quad \mathbf{w} \leftarrow \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \quad \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X}$$

- 损失是凸函数，所以最优解满足

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) &= 0 \\ \Leftrightarrow \frac{1}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X} &= 0 \\ \Leftrightarrow \mathbf{w}^* &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \end{aligned}$$

显示解

- 将偏差加入权重 $\mathbf{X} \leftarrow [\mathbf{X}, \mathbf{1}] \quad \mathbf{w} \leftarrow \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \quad \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X}$$

- 损失是凸函数，所以最优解满足

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) &= 0 \\ \Leftrightarrow \frac{1}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X} &= 0 \\ \Leftrightarrow \mathbf{w}^* &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \end{aligned}$$

【总结】

- 线性回归是对 n 维输入的加权，外加偏差
- 使用平方损失来衡量预测值和真实值的差异
- 线性回归有显示解
- 线性回归可以看做是单层神经网络

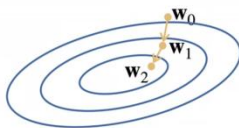
● 基础优化算法

梯度下降

- 挑选一个初始值 $\mathbf{w}_0$
- 重复迭代参数 $t=1, 2, 3$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$$

- 沿梯度方向将增加损失函数值
- 学习率：步长的超参数

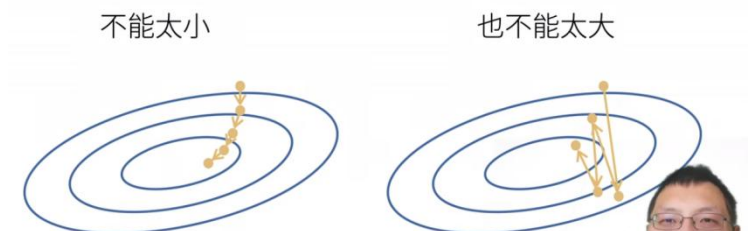


梯度：使得函数的值增加最快的地方

整个代表的是 $\mathbf{w}_0$ 到 $\mathbf{w}_1$ 的向量

学习率：步长，超参数

选择学习率



太小：每次走的步长有限，到达一个点需要走非常多的步骤，计算梯度是一个很贵的事情
太大：步子迈太大一直在震荡，并没有真正在下降

梯度下降里面每次计算梯度要对整个损失函数求导，损失函数是对所有样本的平均损失，所以意味着求一次梯度要把整个样本重新算一遍，这个是很贵的事情，可能需要数分钟或数小时

——我们可以随机采样 b 个样本 $i_1, i_2, \dots, i_b$ 来近似损失

$$\frac{1}{b} \sum_{i \in I_b} \ell(\mathbf{x}_i, y_i, \mathbf{w})$$

- b 是批量大小，另一个重要的超参数

选择批量大小

不能太小

每次计算量太小，不适合并行来最大利用计算资源

不能太大

内存消耗增加浪费计算，例如如果所有样本都是相同的

【总结】

- 梯度下降通过不断沿着反梯度方向更新参数求解
- 小批量随机梯度下降是深度学习默认的求解算法
- 两个重要的超参数是批量大小和学习率

线性回归的简洁实现

通过使用深度学习框架来简洁地实现 线性回归模型 生成数据集

https://courses.d2l.ai/zh-v2/assets/notebooks/chapter_linear-networks/linear-regression-concise.slides.html#/

2、Softmax 回归 + 损失函数 + 图片分类数据集

2.1 【softmax 回归】

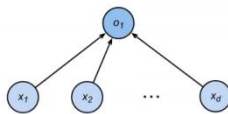
回归 vs 分类

- 回归估计一个连续值
- 分类预测一个离散类别

从回归到多类分类

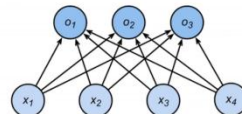
回归

- 单连续数值输出
- 自然区间 $\mathbb{R}$
- 跟真实值的区别作为损失



分类

- 通常多个输出
- 输出 i 是预测为第 i 类的置信度



从回归到多类分类 — 均方损失

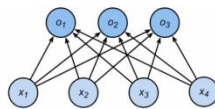
- 对类别进行一位有效编码

$$\mathbf{y} = [y_1, y_2, \dots, y_n]^T$$

$$y_i = \begin{cases} 1 & \text{if } i = y \\ 0 & \text{otherwise} \end{cases}$$

- 使用均方损失训练
- 最大值最为预测

$$\hat{y} = \underset{i}{\operatorname{argmax}} o_i$$



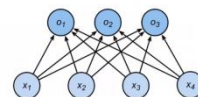
从回归到多类分类 — 无校验比例

- 对类别进行一位有效编码
- 最大值最为预测

$$\hat{y} = \underset{i}{\operatorname{argmax}} o_i$$

- 需要更置信的识别正确类（大余量）

$$o_y - o_i \geq \Delta(y, i)$$



分类不关心实际的值，关心的是不是对真实类别的置信度特别大

——模型真正能够将真正的类和不一样的类拉开距离

——希望使得输出的值是一个概率，引入新的操作子叫 softmax

指数的好处是无论什么值都能变成非负

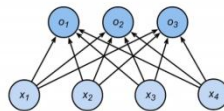
从回归到多类分类 — 校验比例

- 输出匹配概率（非负，和为 1）

$$\hat{\mathbf{y}} = \operatorname{softmax}(\mathbf{o})$$

$$\hat{y}_i = \frac{\exp(o_i)}{\sum_k \exp(o_k)}$$

- 概率 $\mathbf{y}$ 和 $\hat{\mathbf{y}}$ 的区别作为损失



Softmax 和交叉熵损失

交叉熵常用来衡量两个概率的区别 $H(\mathbf{p}, \mathbf{q}) = \sum_i -p_i \log(q_i)$

- 将它作为损失

$$l(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_i y_i \log \hat{y}_i = - \log \hat{y}_y$$

- 其梯度是真实概率和预测概率的区别

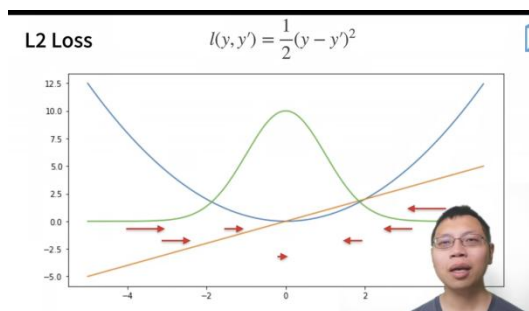
$$\partial_{o_i} l(\mathbf{y}, \hat{\mathbf{y}}) = \operatorname{softmax}(\mathbf{o})_i - y_i$$

总结

- Softmax 回归是一个多类分类模型
- 使用 Softmax 操作子得到每个类的预测置信度
- 使用交叉熵来衡量预测和标号的差别

2.2 损失函数

1、L2 loss 均方损失



蓝色曲线：当 $y=0$ 时，变化预测值 y' 的函数

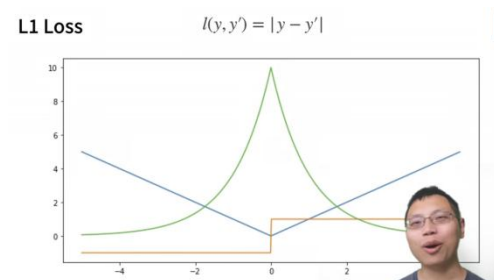
绿色函数：似然函数——1 的 -L 次方

橙色：损失函数的梯度

——导数决定如何更新参数：当预测值 y' 和真实值 y 隔的比较远的时候，梯度比较大，对参数的更新是比较多的。随着预测值慢慢的靠近真实值（原点），梯度变得越来越小，参数更新的幅度也会变得越来越小

缺点：不一定想要那么大的梯度更新参数

2、L1 loss 绝对值损失



蓝色曲线：损失函数曲线

绿色的线：似然函数

橙色：导数

——当 $y' > 0$ 的时候，导数是 1，——即常数 1

当 $y < 0$ 的时候，导数是 -1

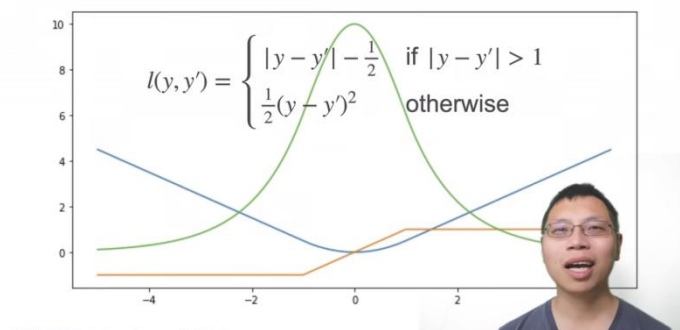
常数在 0 点处不可导

——特性：当预测值和真实值隔的比较远的时候，不管多远，梯度永远是常数，所以就算隔很远，权重分心不会特别大，会带来稳定性的好处

——不好的地方：零点处不可导，当预测值和真实值靠的比较近的时候，会变得不那么稳定

——》提出新的损失函数结合以上两个的好处，避免劣处

Huber's Robust Loss



定义：当预测值和真实值差的比较大的时候，大于1的时候，是绝对值误差

当预测值和真实值靠的比較近的时候，小于等于1的时候，是平方误差

好处：当预测值和真实值差的比較远的时候，梯度用比較均匀的力度往回拉，在近的时候，即优化比較末期的时候，梯度的绝对值会越来越小，保证优化比較平滑，不会出现太多数值上的问题

——分析里预测值比較远的时候和离预测值比較近的作用通过函数形状(梯度形状)来分析特性

三、多层感知机

1、感知机

感知机

• 给定输入 $\mathbf{x}$ ，权重 $\mathbf{w}$ ，和偏移 b ，感知机输出：

$$o = \sigma(\langle \mathbf{w}, \mathbf{x} \rangle + b) \quad \sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{otherwise} \end{cases}$$

• 二分类：-1 或 1

• Vs. 回归输出实数

• Vs. Softmax 回归输出概率



训练感知机

```
initialize  $w = 0$  and  $b = 0$ 
repeat  $\neg 1 > 0 \leq 0$ 
  if  $y_i [\langle w, x_i \rangle + b] \leq 0$  then
     $w \leftarrow w + y_i x_i$  and  $b \leftarrow b + y_i$ 
  end if
until all classified correctly
```

等价于使用批量大小为1的梯度下降，
并使用如下的损失函数

$$\ell(y, \mathbf{x}, \mathbf{w}) = \max(0, -y\langle \mathbf{w}, \mathbf{x} \rangle)$$

负负、正正都为正

批量大小为1——每次拿1个样本算梯度进行更新

Max0——分类正确大于0，分类错误有梯度在上面 if 语句进行更新

收敛定理：什么时候能够停

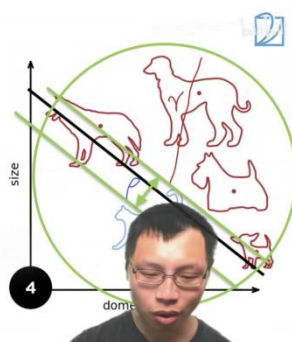
收敛定理

- 数据在半径 r 内
- 余量 ρ 分类两类

$$y(\mathbf{x}^T \mathbf{w} + b) \geq \rho$$

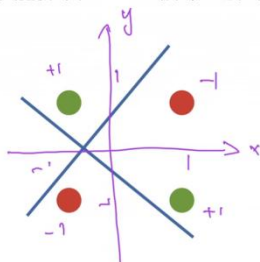
对于 $\|\mathbf{w}\|^2 + b^2 \leq 1$

- 感知机保证在 $\frac{r^2 + 1}{\rho^2}$ 步后收敛



XOR 问题 (Minsky & Papert, 1969)

感知机不能拟合 XOR 函数，它只能产生线性分割面



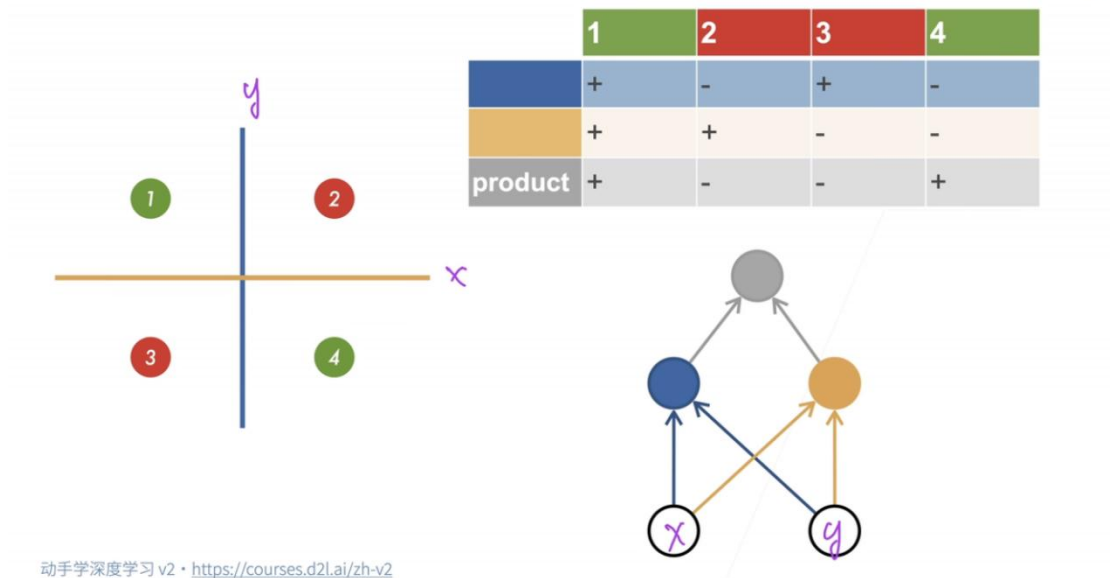
感知机的分割面是一条线，不管怎么切它都无法在一条线把数据给分类，感知机不能拟合 xor 函数，因为他只能产生线性分割面

总结

- 感知机是一个二分类模型，是最早的 AI 模型之一
- 它的求解算法等价于使用批量大小为 1 的梯度下降
- 它不能拟合 XOR 函数，导致的第一次 AI 寒冬

2、【多层感知机】

学习 XOR

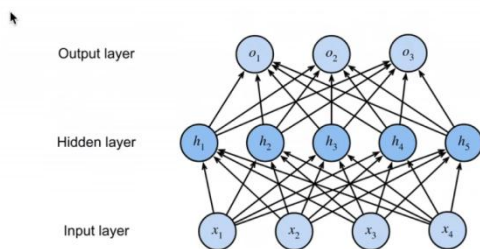


先学习蓝色的线，1、3 为正值，2、4 为负值

再学习橙色的线，1、2 为正值，3、4 为负

有了蓝色的分类器和橙色的分类器，再对两个结果看是否一样，不一样为负，一样为正

单隐藏层



隐藏层大小是超参数

单隐藏层 — 单分类

- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏层 $\mathbf{W}_1 \in \mathbb{R}^{m \times n}, \mathbf{b}_1 \in \mathbb{R}^m$
- 输出层 $\mathbf{w}_2 \in \mathbb{R}^m, b_2 \in \mathbb{R}$

$$\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$o = \mathbf{w}_2^T \mathbf{h} + b_2$$

σ 是按元素的激活函数

单隐藏层

- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏层 $\mathbf{W}_1 \in \mathbb{R}^{m \times n}, \mathbf{b}_1 \in \mathbb{R}^m$
- 输出层 $\mathbf{w}_2 \in \mathbb{R}^m, b_2 \in \mathbb{R}$

$$\mathbf{h} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1 \quad \sigma(x) = \text{ReLU}$$

$$o = \mathbf{w}_2^T \mathbf{h} + b_2$$

$$\text{hence } o = \mathbf{w}_2^T \mathbf{W}_1 \mathbf{x} + b'$$

$$\mathbf{w}_2^T \mathbf{W}_1$$

仍然是线性

为什么需要非线性
激活函数?

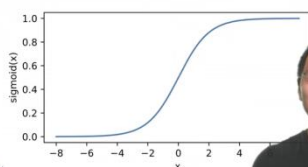
动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>

线性函数——等价于单层感知机
——激活函数要是非线性函数

Sigmoid 激活函数

将输入投影到 $(0, 1)$, 是一个软的 $\sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$

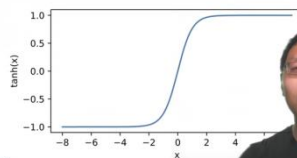
$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}$$



Tanh 激活函数

将输入投影到 $(-1, 1)$

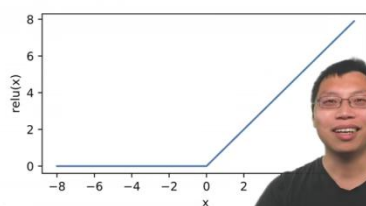
$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}$$



ReLU 激活函数

ReLU: rectified linear unit

$$\text{ReLU}(x) = \max(x, 0)$$



指数运算是件很贵的事情, max 搞定简单

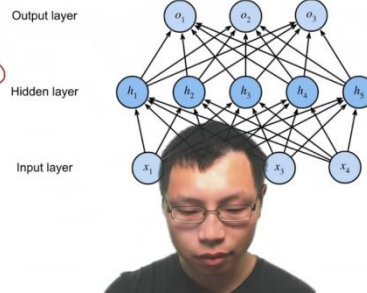
多类分类

- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏层 $\mathbf{W}_1 \in \mathbb{R}^{m \times n}, \mathbf{b}_1 \in \mathbb{R}^m$
- 输出层 $\mathbf{W}_2 \in \mathbb{R}^{m \times k}, \mathbf{b}_2 \in \mathbb{R}^k$

$$\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{W}_2^T \mathbf{h} + \mathbf{b}_2$$

$$\mathbf{y} = \text{softmax}(\mathbf{o})$$



多类分类和单分类的区别——多类分类多 o 要多做一次 softmax
最后一层不需要激活函数，隐藏层激活函数不能少
——激活函数：避免层数的塌陷，输出不需要激活函数

多隐藏层

$$\mathbf{h}_1 = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

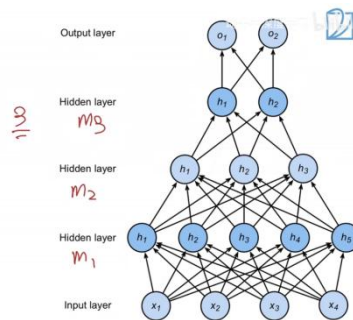
$$\mathbf{h}_2 = \sigma(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2)$$

$$\mathbf{h}_3 = \sigma(\mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3)$$

$$\mathbf{o} = \mathbf{W}_4 \mathbf{h}_3 + \mathbf{b}_4$$

超参数

- 隐藏层数
- 每层隐藏层的大小



最好 $m_1 > m_2 > m_3$

总结

- 多层感知机使用隐藏层和激活函数来得到非线性模型
- 常用激活函数是 Sigmoid, Tanh, ReLU
- 使用 Softmax 来处理多类分类
- 超参数为隐藏层数，和各个隐藏层大小

3、模型选择

训练误差和泛化误差

- 训练误差：模型在训练数据上的误差
- 泛化误差：模型在新数据上的误差
- 例子：根据模考成绩来预测未来考试分数
 - 在过去的考试中表现很好（训练误差）不代表未来考试一定会好（泛化误差）
 - 学生 A 通过背书在模考中拿到很好成绩
 - 学生 B 知道答案后面的原因



验证数据集和测试数据集

- 验证数据集：一个用来评估模型好坏的数据集
 - 例如拿出 50% 的训练数据
 - 不要跟训练数据混在一起（常犯错误）
- 测试数据集：只用一次的数据集。例如
 - 未来的考试
 - 我出价的房子的实际成交价
 - 用在 Kaggle 私有排行榜中的数据



验证数据集：反映超参数的选择好坏，可以用来调超参数

测试数据集：只能被使用一次

——50% 训练，50% 验证

——遇到的问题：没有足够多数据时使用，样本量小的时候，只拿一半训练模型会比较亏

——常见解决方法：K-则交叉验证

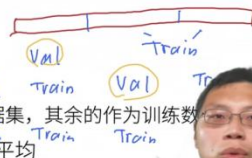
——尽可能尽大的使用数据集作为训练数据集进行训练，代价是算起来特别贵，5 就是跑 5 次

——常见一般是取 5/10，数据很大取 2/3，数据很小取 10 或更高

——k 是一个权衡要用多少数据进行训练以及能承受多少次/多少倍的训练的代价

K-则交叉验证

- 在没有足够多数据时使用（这是常态）
- 算法：
 - 将训练数据分割成 K 块
 - For $i = 1, \dots, K$
 - 使用第 i 块作为验证数据集，其余的作为训练数据
 - 报告 K 个验证集误差的平均
- 常用：K = 5 或 10



总结

- 训练数据集：训练模型参数
- 验证数据集：选择模型超参数
- 非大数据集上通常使用 k-折交叉验证

——通过 k 次的训练来判断超参数的好坏，比如有 10 种不同的超参数（隐藏大小、学习率等），把最好的精度选出来作为要采用的超参数

4、过拟合和欠拟合

过拟合和欠拟合



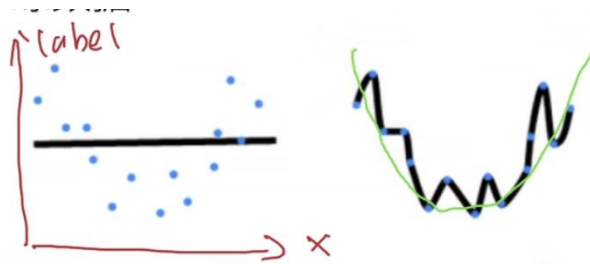
		数据	
		简单	复杂
模型容量	低	正常	欠拟合
	高	过拟合	正常

模型容量就是模型复杂度：如线性模型就会比较简单，多层感知机就会比较复杂
数据：简单数据、复杂数据

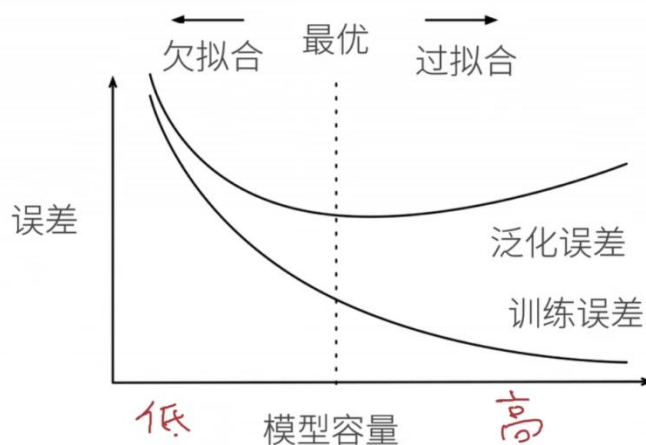
——根据数据集的简单和复杂选择对应的模型容量：数据简单选择比较低的模型容量；数据比较复杂选择比较高的模型容量

模型容量定义：拟合各种函数的能力

- 低容量的模型难以拟合训练数据
- 高容量的模型可以记住所有的训练数据



模型容量的影响



【估计模型容量】

- 难以在不同的种类算法之间比较
 - 例如树模型和神经网络
- 给定一个模型种类，将有两个主要因素
 - 参数的个数
 - 参数值的选择范围

VC 维

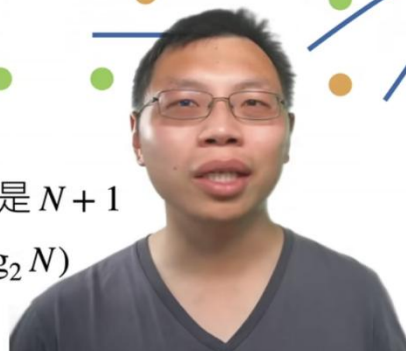
- 统计学习理论的一个核心思想
- 对于一个分类模型，VC 等于一个最大的数据集的大小，不管如何给定标号，都存在一个模型来对它进行完美分类

线性分类器的 VC 维

- 2 维输入的感知机，VC 维 = 3
 - 能够分类任何三个点，但不是4个 (xor)



- 支持 N 维输入的感知机的 VC 维是 $N + 1$
- 一些多层感知机的 VC 维 $O(N \log_2 N)$



动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>

VC 维的用处

提供为什么一个模型好的理论依据

- 它可以衡量训练误差和泛化误差之间的间隔

但深度学习中很少使用

- 衡量不是很准确
- 计算深度学习模型的 VC 维很困难

数据复杂度

- 多个重要因素
 - 样本个数
 - 每个样本的元素个数（是二维向量、还是很小图片）
 - 时间、空间结构（图片、视频）
 - 多样性（类里面是做 10 类还是 100 类的分类）

【总结】

- **模型容量需要匹配数据复杂度**，否则可能导致欠拟合和过拟合
- 统计机器学习提供数学工具来衡量模型复杂度
- 实际中一般靠观察训练误差和验证误差

5、权重衰退 weight_decay

——最常见处理过拟合的方法

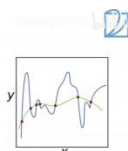
1、

使用均方范数作为硬性限制

- 通过限制参数值的选择范围来控制模型容量

$$\min \ell(\mathbf{w}, b) \quad \text{subject to} \quad \|\mathbf{w}\|^2 \leq \theta$$

- 通常不限制偏移 b (限不限制差不多)
- 小的 θ 意味着更强的正则项



在最小化损失函数的时候加入一个限制， w 的平方小于一个特定的值

2、

使用均方范数作为柔性限制

- 对每个 θ ，都可以找到 λ 使得之前的目标函数等价于下面

$$\min \ell(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

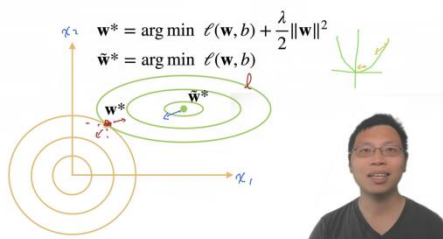
- 可以通过拉格朗日乘子来证明
- 超参数 λ 控制了正则项的重要程度
- $\lambda = 0$: 无作用
- $\lambda \rightarrow \infty, \mathbf{w}^* \rightarrow \mathbf{0}$



定义 L_2 范数惩罚

```
In [4]: def l2_penalty(w):  
        return torch.sum(w.pow(2)) / 2
```

演示对最优解的影响



罚的引入使得最优解往原点走了

参数更新法则

- 计算梯度

$$\frac{\partial}{\partial \mathbf{w}} \left(\ell(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2 \right) = \frac{\partial \ell(\mathbf{w}, b)}{\partial \mathbf{w}} + \lambda \mathbf{w}$$

- 时间 t 更新参数 $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\partial \ell}{\partial \mathbf{w}_t}$

$$\mathbf{w}_{t+1} = (1 - \eta \lambda) \mathbf{w}_t - \eta \frac{\partial \ell(\mathbf{w}_t, b_t)}{\partial \mathbf{w}_t}$$

- 通常 $\eta \lambda < 1$ ，在深度学习中通常叫做权重衰退

总结

- 权重衰退通过 L2 正则项使得模型参数不会过大，从而控制模型复杂度
- 正则项权重是控制模型复杂度的超参数

6、丢弃法 dropout

动机

- 一个好的模型需要对输入数据的扰动鲁棒
 - 使用有噪音的数据等价于 Tikhonov 正则（正则就是使得权重不要特别大，都可以认为是避免过拟合的方法）
 - 丢弃法：在层之间加入噪音

无偏差的加入噪音

- 对 $\mathbf{x}$ 加入噪音得到 $\mathbf{x}'$ ，我们希望 $E[x'_i] = p \cdot 0 + (1-p) \frac{x_i}{1-p} = x_i$

$$E[\mathbf{x}'] = \mathbf{x}$$

- 丢弃法对每个元素进行如下扰动

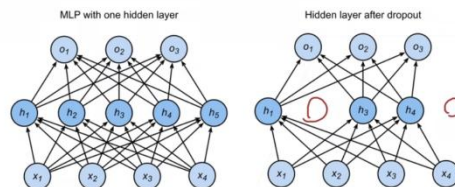
$$x'_i = \begin{cases} 0 & \text{with probability } p \\ \frac{x_i}{1-p} & \text{otherwise} \end{cases}$$



使用丢弃法

- 通常将丢弃法作用在隐藏全连接层的输出上

$$\begin{aligned} \mathbf{h} &= \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \\ \mathbf{h}' &= \text{dropout}(\mathbf{h}) \\ \mathbf{o} &= \mathbf{W}_2 \mathbf{h}' + \mathbf{b}_2 \\ \mathbf{y} &= \text{softmax}(\mathbf{o}) \end{aligned}$$



$\mathbf{h}$ 是第一个隐藏层的输出

推理中的丢弃法

- 正则项只在训练中使用：他们影响模型参数的更新
 - 在推理过程中，丢弃法直接返回输入
- $\mathbf{h} = \text{dropout}(\mathbf{h})$
- 这样也能保证确定性的输出

总结

- 丢弃法将一些输出项随机置 0 来控制模型复杂度
- 常作用在多层感知机的隐藏层输出上
- 丢弃概率是控制模型复杂度的超参数

dropout 现在是最主流的对于多层感知机模型控制的方法

可以把模型弄的复杂一点点，然后通过正则化控制模型复杂度

7、数值稳定性

神经网络的梯度

- 考虑如下有 d 层的神经网络

$$\mathbf{h}^t = f_t(\mathbf{h}^{t-1}) \quad \text{and} \quad y = \ell \circ f_d \circ \dots \circ f_1(\mathbf{x})$$

- 计算损失 ℓ 关于参数 $\mathbf{W}_t$ 的梯度

$$\frac{\partial \ell}{\partial \mathbf{W}^t} = \frac{\partial \ell}{\partial \mathbf{h}^d} \underbrace{\frac{\partial \mathbf{h}^d}{\partial \mathbf{h}^{d-1}} \dots \frac{\partial \mathbf{h}^{t+1}}{\partial \mathbf{h}^t}}_{d-t \text{ 次矩阵乘法}} \frac{\partial \mathbf{h}^t}{\partial \mathbf{W}^t}$$

动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>

$d-t$ 次矩阵乘法

$\mathbf{h}$ 都是向量，向量关于向量的导数是一个矩阵

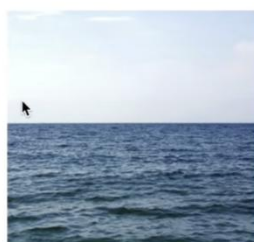
——» 数值稳定性会带来常见两个问题

梯度爆炸



$$1.5^{100} \approx 4 \times 10^{17}$$

梯度消失



$$0.8^{100} \approx 2 \times 10^{-10}$$

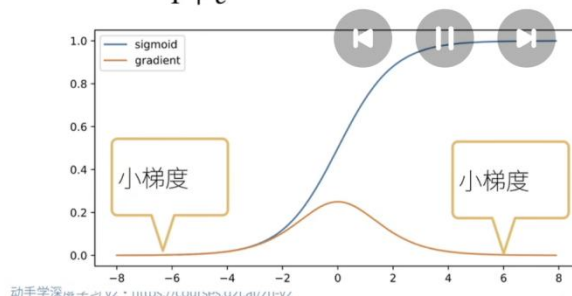
梯度爆炸的问题

- 值超出值域 (infinity)
 - 对于 16 位浮点数尤为严重 (数值区间 $6e-5$ - $6e4$)
- 对学习率敏感
 - 如果学习率太大——» 大参数值——» 更大的梯度
 - 如果学习率太小——» 训练无进展
 - 我们可能需要在训练过程不断调整学习率

梯度消失

- 使用 sigmoid 作为激活函数

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$



梯度消失的问题：梯度为 0，不管学习率为多少都不会有进展（因为权重是学习率*梯度）
——不管把神经网络架的多深，底部那些层跑不动，只训练了顶部那些层，和一个很浅的神经网络没什么区别

梯度消失的问题

- 梯度值变成 0
- 对 16 位浮点数尤为严重
- 训练没有进展
 - 不管如何选择学习率
- 对于底部层尤为严重
 - 仅仅顶部层训练的较好
 - 无法让神经网络更深

总结

- 当数值过大或者过小时会导致数值问题
- 常发生在深度模型中，因为其会对 n 个数累乘

8、模型稳定性和激活函数

让训练更加稳定

- 目标：让梯度值在合理的范围内
 - 例如 $[1e-6, 1e3]$
- 将乘法变加法
 - ResNet, LSTM
- 归一化
 - 梯度归一化，梯度裁剪

•合理的权重初始和激活函数

其中一个想法：

让每层的方差是一个常数

- 将每层的输出和梯度都看做随机变量
- 让它们的均值和方差都保持一致

$$\begin{array}{cc} \text{正向} & \text{反向} \\ \mathbb{E}[h_i^t] = 0 & \mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial h_i^t}\right] = 0 \\ \text{Var}[h_i^t] = a & \text{Var}\left[\frac{\partial \mathcal{L}}{\partial h_i^t}\right] = b \quad \forall i, t \end{array}$$

a 和 b 都是常数

权重初始化

- 在合理值区间里随机初始参数
- 训练开始的时候更容易有数值不稳定
 - 远离最优解的地方损失函数表面可能很复杂
 - 最优解附近表面会比较平
- 使用 $\mathcal{N}(0, 0.01)$ 来初始可能对小网络没问题，但不能保证深度神经网络



例子：MLP

- 假设
 - $w_{i,j}^t$ 是 i.i.d, 那么 $\mathbb{E}[w_{i,j}^t] = 0$, $\text{Var}[w_{i,j}^t] = \gamma_t$
 - h_i^{t-1} 独立于 $w_{i,j}^t$
- 假设没有激活函数 $\mathbf{h}^t = \mathbf{W}^t \mathbf{h}^{t-1}$, 这里 $\mathbf{W}^t \in \mathbb{R}^{n_t \times n_{t-1}}$

$$\mathbb{E}[h_i^t] = \mathbb{E}\left[\sum_j w_{i,j}^t h_j^{t-1}\right] = \sum_j \mathbb{E}[w_{i,j}^t] \mathbb{E}[h_j^{t-1}] = 0$$

当前层输入独立于当前层权重



正向方差

$$\begin{aligned}
 \text{Var}[h_i^t] &= \mathbb{E}[(h_i^t)^2] - \mathbb{E}[h_i^t]^2 = \mathbb{E}\left[\left(\sum_j w_{i,j}^t h_j^{t-1}\right)^2\right] \\
 &= \mathbb{E}\left[\sum_j (w_{i,j}^t)^2 (h_j^{t-1})^2 + \sum_{j \neq k} w_{i,j}^t w_{i,k}^t h_j^{t-1} h_k^{t-1}\right] \quad \text{= 0} \\
 &= \sum_j \mathbb{E}\left[(w_{i,j}^t)^2\right] \mathbb{E}\left[(h_j^{t-1})^2\right] \quad \text{目标} \quad n_{t-1}\gamma_t = 1 \\
 &= \sum_j \text{Var}[w_{i,j}^t] \text{Var}[h_j^{t-1}] = n_{t-1}\gamma_t \text{Var}[h_j^{t-1}]
 \end{aligned}$$

目标是输入的方差和输出的方差一样

反向均值和方差

• 跟正向情况类似

$$\frac{\partial \ell}{\partial \mathbf{h}^{t-1}} = \frac{\partial \ell}{\partial \mathbf{h}^t} \mathbf{W}^t \quad \Rightarrow \quad \left(\frac{\partial \ell}{\partial \mathbf{h}^{t-1}}\right)^T = (\mathbf{W}^t)^T \left(\frac{\partial \ell}{\partial \mathbf{h}^t}\right)^T$$

$$\mathbb{E}\left[\frac{\partial \ell}{\partial h_i^{t-1}}\right] = 0$$

$$\text{Var}\left[\frac{\partial \ell}{\partial h_i^{t-1}}\right] = n_t \gamma_t \text{Var}\left[\frac{\partial \ell}{\partial h_j^t}\right] \quad \Rightarrow \quad n_t \gamma_t = 1$$

n_{t-1} 是第 t 层输入的维度, n_t 是第 t 层输出的维度 (除非输入=输出, 不然很难同时满足这两个条件)

伽马 γ_t 是第 t 层权重的方差

xavier 是常用的权重初始化的方法

Xavier 初始

- 难以需要满足 $n_{t-1}\gamma_t = 1$ 和 $n_t\gamma_t = 1$
- Xavier 使得 $\gamma_t(n_{t-1} + n_t)/2 = 1 \rightarrow \gamma_t = 2/(n_{t-1} + n_t)$
 - 正态分布 $\mathcal{N}(0, \sqrt{2/(n_{t-1} + n_t)})$
 - 均匀分布 $\mathcal{U}(-\sqrt{6/(n_{t-1} + n_t)}, \sqrt{6/(n_{t-1} + n_t)})$
 - 分布 $\mathcal{U}[-a, a]$ 和方差是 $a^2/3$
- 适配权重形状变换, 特别是 n_t



假设线性的激活函数

- 假设 $\sigma(x) = \alpha x + \beta$

$$\mathbf{h}' = \mathbf{W}^t \mathbf{h}^{t-1} \quad \text{and} \quad \mathbf{h}^t = \sigma(\mathbf{h}')$$

$$\mathbb{E}[h'_i] = \mathbb{E}[\alpha h'_i + \beta] = \beta \quad \Rightarrow \quad \beta = 0$$

$$\begin{aligned} \text{Var}[h'_i] &= \mathbb{E}[(h'_i)^2] - \mathbb{E}[h'_i]^2 \\ &= \mathbb{E}[(\alpha h'_i + \beta)^2] - \beta^2 \quad \Rightarrow \quad \alpha = 1 \\ &= \mathbb{E}[\alpha^2 (h'_i)^2 + 2\alpha\beta h'_i + \beta^2] - \beta^2 \\ &= \alpha^2 \text{Var}[h'_i] \end{aligned}$$

假设线性的激活函数

- 假设 $\sigma(x) = \alpha x + \beta$

$$\mathbf{h}' = \mathbf{W}^t \mathbf{h}^{t-1} \quad \text{and} \quad \mathbf{h}^t = \sigma(\mathbf{h}')$$

$$\mathbb{E}[h'_i] = \mathbb{E}[\alpha h'_i + \beta] = \beta \quad \Rightarrow \quad \beta = 0$$

$$\begin{aligned} \text{Var}[h'_i] &= \mathbb{E}[(h'_i)^2] - \mathbb{E}[h'_i]^2 \\ &= \mathbb{E}[(\alpha h'_i + \beta)^2] - \beta^2 \quad \Rightarrow \quad \alpha = 1 \\ &= \mathbb{E}[\alpha^2 (h'_i)^2 + 2\alpha\beta h'_i + \beta^2] - \beta^2 \\ &= \alpha^2 \text{Var}[h'_i] \end{aligned}$$

检查常用激活函数

- 使用泰勒展开

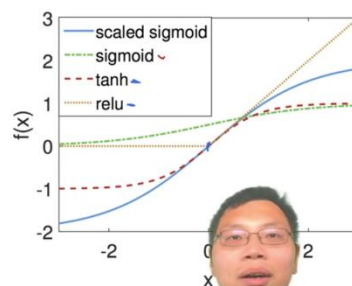
$$\text{sigmoid}(x) = \frac{1}{2} + \frac{x}{4} - \frac{x^3}{48} + O(x^5)$$

$$\tanh(x) = 0 + x - \frac{x^3}{3} + O(x^5)$$

$$\text{relu}(x) = 0 + x \quad \text{for } x \geq 0$$

- 调整 sigmoid:

$$4 \times \text{sigmoid}(x) - 2$$



总结

- 合理的权重初始值和激活函数的选取可以提升数值稳定性

四、循环神经网络

1、序列数据

很多数据是有时序结构的

Eg: 电影的评价随时间变化而变化

序列数据 - 更多例子

- 音乐、语言、文本、和视频都是连续的
 - 标题“狗咬人”远没有“人咬狗”那么令人惊讶
- 大地震发生后，很可能会有几次较小的余震
- 人的互动是连续的，从网上吵架可以看出
- 预测明天的股价要比填补昨天遗失的股份的更困难



统计工具

- 在时间 t 观察到 x_t ，那么得到 T 个不独立的随机变量 $(x_1, \dots, x_T) \sim p(\mathbf{x})$
- 使用条件概率展开 $p(a, b) = p(a)p(b|a) = p(b)p(a|b)$



序列模型

$$p(\mathbf{x}) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots \cdot p(x_T | x_1, \dots, x_{T-1})$$



- 对条件概率建模

$$p(x_t | x_1, \dots, x_{t-1}) = p(x_t | f(x_1, \dots, x_{t-1}))$$

对见过的数据建模，也称
自回归模型



对前面 t 个进行建模，把他表示称 f 函数，可以认为是一个机器学习模型

——在过去的 $t-1$ 个数据上训练一个模型，然后来预测下一个模型（标号和真实的样本是一个东西）

——自回归模型

——怎么算 p

方案 A - 马尔科夫假设

$$p(\mathbf{x}) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots \cdot p(x_T | x_1, \dots, x_{T-1})$$



- 假设当前数据只跟 τ 个过去数据点相关



$$p(x_t | x_1, \dots, x_{t-1}) = p(x_t | x_{t-\tau}, \dots, x_{t-1}) = p(x_t | f(x_{t-\tau}, \dots, x_{t-1}))$$

例如在过去数据上训练
一个MLP模型

动手学深度学习 v2 · <https://course>

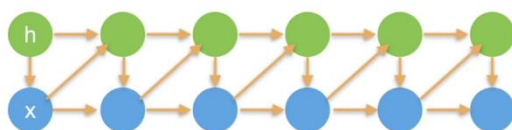


方案 B - 潜变量模型

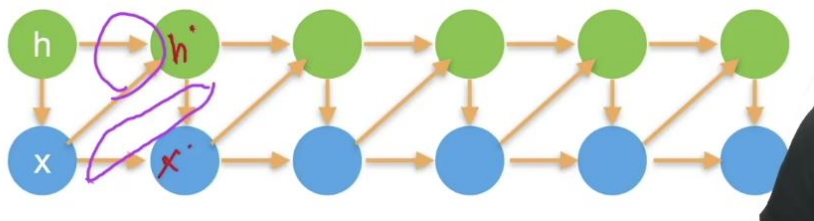
$$p(\mathbf{x}) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots \cdot p(x_T | x_1, \dots, x_{T-1})$$



- 引入潜变量 h_t 来表示过去信息 $h_t = f(x_1, \dots, x_{t-1})$
- 这样 $x_t = p(x_t | h_t)$



动手学深度学习 v2 · <https://course>



——» 算 x' 和前面的 x 相关也和因变量 h' 相关

——» h' 和前面时间的前变量和前面的 x 相关

——等价于建两个模型

一个模型是怎么根据前面时间的 h 和 x 算新的潜变量 h'

另一个模型：给定新的潜变量的状态和前面的 x 算新的 x'

总结

- 时序模型中，当前数据跟之前观察到的数据相关
- 自回归模型使用自身过去数据来预测未来
- 马尔科夫模型假设当前只跟最近少数数据相关，从而简化模型
- 潜变量模型使用潜变量来概括历史信息

——» 代码：用马尔科夫假设来训练 mlp 怎么来预测模型
——mlp 难点是预测很远的未来

代码：

每个文本序列拆分成标记列表 (token-分词)

构建一个字典，通常也叫做词汇表 (vocabulary)，用来将字符串类型的标记映射到从 0 开始的数字索引中

将每一条文本行转换成一个数字索引列

将所有功能大报道 load_corpus_time_machine 函数中

2、语言模型

预测整个文本序列出现的概率

语言模型



- 给定文本序列 $x_1, \dots, x_T$ ，语言模型的目标是估计联合概率

$$p(x_1, \dots, x_T)$$

- 它的应用包括

- 做预训练模型 (eg BERT, GPT-3)
- 生成文本，给定前面几个词，不断的使用 $x_t \sim p(x_t | x_1, \dots, x_{t-1})$ 来生成后续文本
- 判断多个序列中哪个更常见，e.g. “to recognize speech” vs “to wreck a nice beach”

动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>

使用计数来建模



- 假设序列长度为2，我们预测

$$p(x, x') = p(x)p(x'|x) = \frac{n(x)}{n} \frac{n(x, x')}{n(x)}$$

- 这里 n 是总词数， $n(x), n(x, x')$ 是单个单词和连续单词对的出现次数

- 很容易拓展到长为3的情况

$$p(x, x', x'') = p(x)p(x'|x)p(x''|x, x') = \frac{n(x)}{n} \frac{n(x, x')}{n(x)} \frac{n(x, x', x'')}{n(x, x')}$$

N元语法



- 当序列很长时，因为文本量不够大，很可能 $n(x_1, \dots, x_T) \leq 1$

- 使用马尔科夫假设可以缓解这个问题

$$\begin{aligned} p(x_1, x_2, x_3, x_4) &= p(x_1)p(x_2)p(x_3)p(x_4) \\ \text{一元语法:} \quad &= \frac{n(x_1)}{n} \frac{n(x_2)}{n} \frac{n(x_3)}{n} \frac{n(x_4)}{n} \end{aligned}$$

$$\begin{aligned} p(x_1, x_2, x_3, x_4) &= p(x_1)p(x_2|x_1)p(x_3|x_2)p(x_4|x_3) \\ \text{二元语法:} \quad &= \frac{n(x_1)}{n} \frac{n(x_2, x_1)}{n(x_1)} \frac{n(x_3, x_2)}{n(x_2)} \frac{n(x_4, x_3)}{n(x_3)} \end{aligned}$$

- 三元语法: $p(x_1, x_2, x_3, x_4) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2)p(x_4|x_2, x_3)$

序列很长用马尔可夫假设

——n 元语法：把长度为 n 的序列存起来

总结

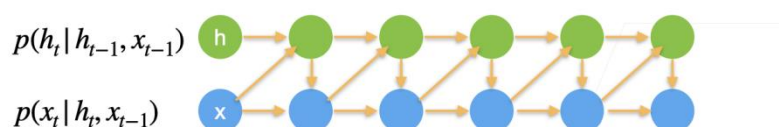
- 语言模型估计文本序列的联合概率
- 使用统计方法时常采用 n 元语法

3、循环神经网络（第一个对于序列模型神经网络） rnn

潜变量自回归模型



- 使用潜变量 h_t 总结过去信息



h_t 和 h_{t-1} 、 x_{t-1} 相关

x_t 和 h_t 、 x_{t-1} 相关

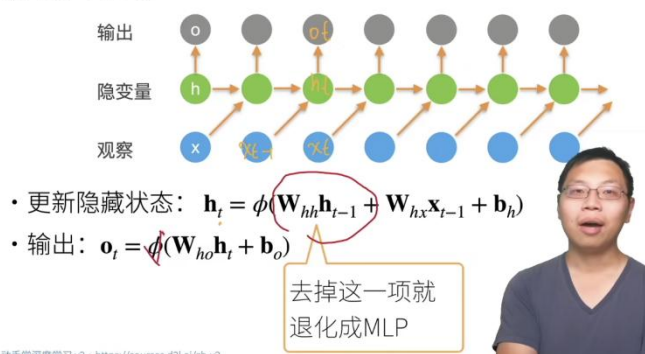
怎么把这个模型变成 rnn

—— o_t 根据 h_t 来输出 (h_t 用的是 x_{t-1} 的东西，计算损失是比较 o_t 和 x_t 的关系)

——当前时刻的输出发生在观察之前

——最简单的 rnn 是通过 whh 来存时序信息的 (和 mlp 比就是通过额外的 whh 来存时序信息)

循环神经网络



怎么样衡量语言模型的好坏

——语言模型是分类模型

——可以用交叉熵

困惑度 (perplexity)

- 衡量一个语言模型的好坏可以用平均交叉熵

$$\pi = \frac{1}{n} \sum_{i=1}^n -\log p(x_i | x_{1:i-1}, \dots)$$

p 是语言模型的预测概率, x_i 是真实词

- 历史原因NLP使用困惑度 $\exp(\pi)$ 来衡量, 是平均每次可能选项
- 1表示完美, 无穷大是最差情况

——perplexity 作完指数, 数值更大了, 在直观上更好理解

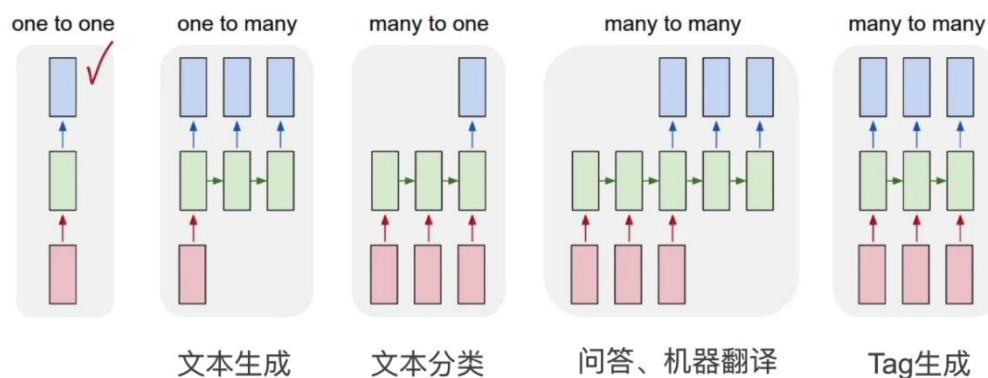
梯度裁剪

- 迭代中计算这 T 个时间步上的梯度, 在反向传播过程中产生长度为 $O(T)$ 的矩阵乘法链, 导致数值不稳定
- 梯度裁剪能有效预防梯度爆炸
- 如果梯度长度超过 θ , 那么拖影回长度 θ

$$\mathbf{g} \leftarrow \min\left(1, \frac{\theta}{\|\mathbf{g}\|}\right) \mathbf{g}$$

——梯度裁剪: 保证值永远不会超过 θ , 有效避免梯度爆炸

更多的应用 RNNs



总结

- 循环神经网络的输出取决于当下输入和前一段时间的隐变量
- 应用到语言模型中时，循环神经网络根据当前词预测下一次时刻词
- 通常使用困惑度来衡量语言模型的好坏

4、门控循环单元 (gru)

——要知道什么叫门控

——时序太长隐藏状态累积了太多东西，对于前面很久东西不太好去抽取出来

——门控通过一些额外的控制单元使得我们在构造隐藏状态的时候可以选择那块比较重要

——有两种门：更新门、重置门

关注一个序列



- 不是每个观察值都是同等重要



- 想只记住相关的观察需要：
 - 能关注的机制（更新门）
 - 能遗忘的机制（重置门）



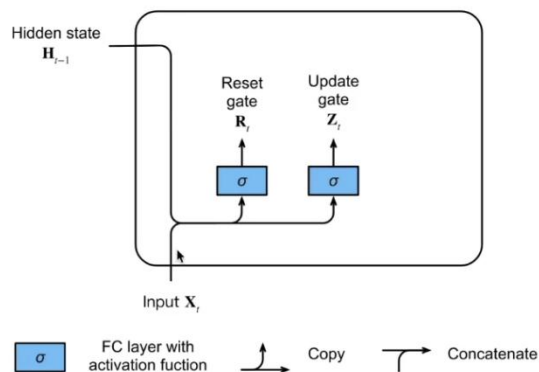
rt 重置

zt 更新

门

$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r),$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$



- sigma 是 sigmoid 的函数
- 等价于使用 sigmoid 算隐藏状态，全连接层
- rt 和 ht 长度一样，元素靠近 0，rt*ht-1 这个东西乘出来就会变得像 0
- 像控制单元（软的）

候选隐状态

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

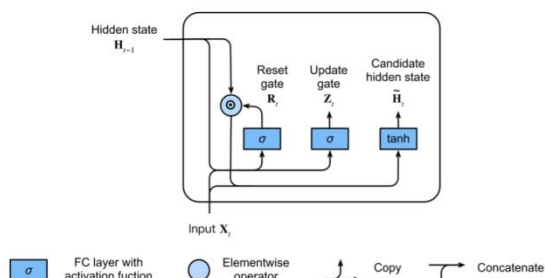


图9.1.2说明了应用重置门之后的计算流程。

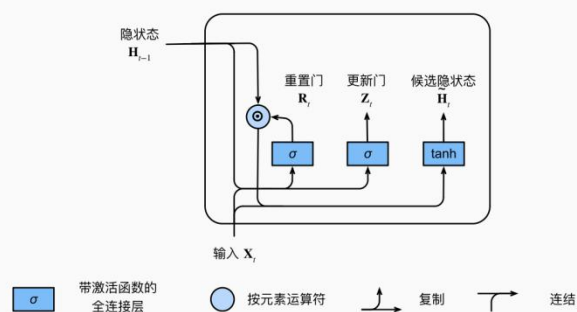


图9.1.2 在门控循环单元模型中计算候选隐状态

隐状态

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$

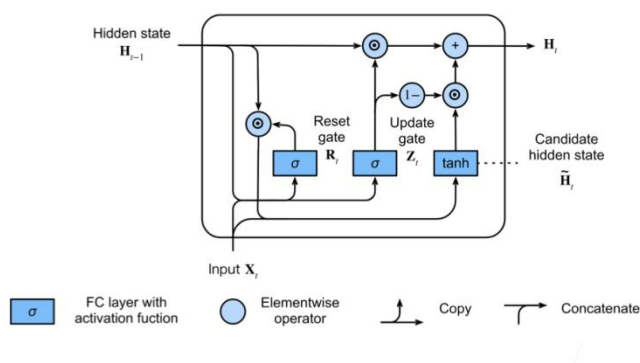


图9.1.3说明了更新门起作用后的计算流。

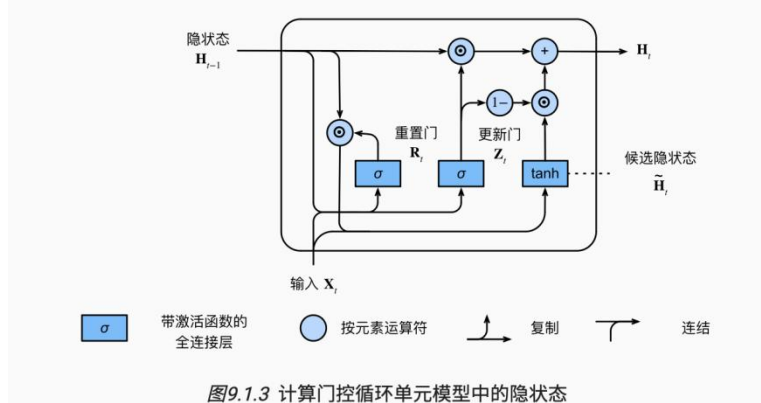


图9.1.3 计算门控循环单元模型中的隐状态

GRU 引入了两个额外的门，每个门可以学的参数和 RNN 一样多，本质上来说可学习的权重是 RNN 的三倍，这两个门是控制单元，输出 0-1 之间的数值，reset 是说更新新的隐藏状态的时候要用到多少过去隐藏状态的信息，另外一个是说算当前隐藏状态的时候需要用多少关于 x_t 的信息（极端情况 $z=0$, $r=1$ 就会回到之前 rnn 的情况，另外一个极端情况是只看 x_t 当下，其他都不管了）——加入这两个门就是在这两个极端情况之间调整，到底是多去看现在还是过去的信息

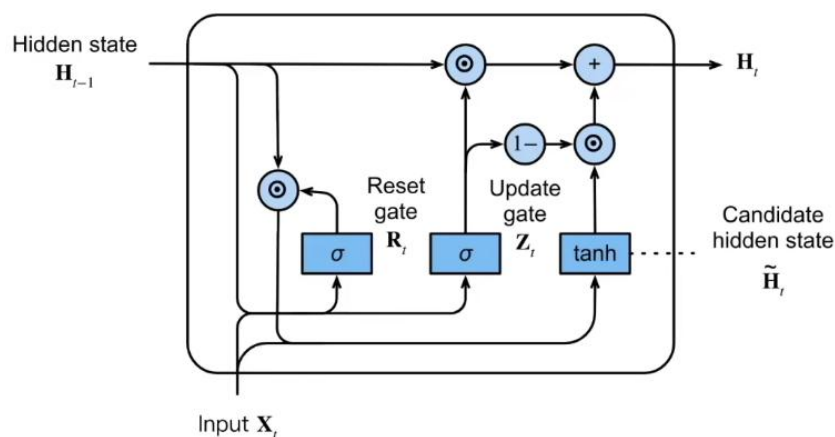
总结

$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r),$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$



5、长短期记忆网络 (LSTM)

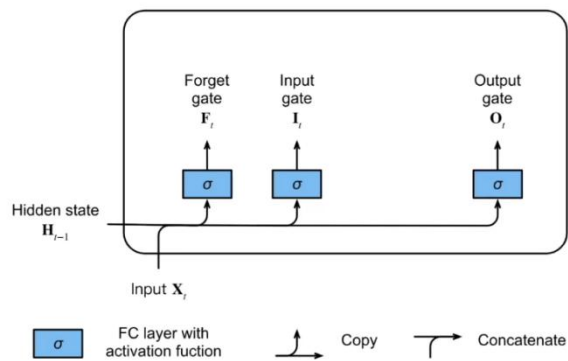
要实现的效果和 GRU 是一样的，要用的东西比 GRU 复杂

效果上来讲都是是不是要忘掉过去的状态，去看现在的输入，还是不看现在的输入，用前一个时刻的状态

长短期记忆网络

- 忘记门：将值朝0减少
- 输入门：决定不是忽略掉输入数据
- 输出门：决定是不是使用隐状态

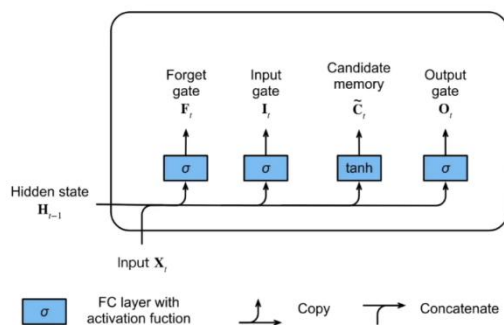
$$\begin{aligned}
 I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \\
 F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \\
 O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)
 \end{aligned}$$



候选记忆单元

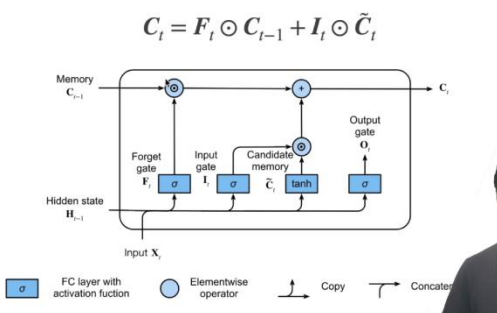
候选记忆单元

$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$



Lstm 和 RNN、gru 不一样的地方：上个时刻的记忆单元会作为状态放进来。
lstm 状态有两个：c 和 h

记忆单元

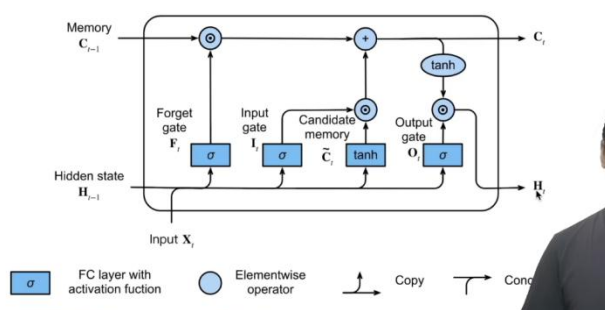


tanh 是把东西变到了-1 到 1 之间

ot 是要不要输出，0 是不要输出，1 是输出了

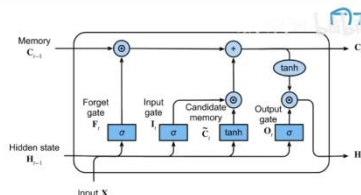
隐状态

$$H_t = O_t \odot \tanh(C_t)$$



总结

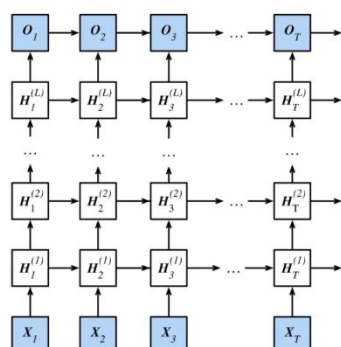
$$\begin{aligned} I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \\ F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \\ O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \\ \tilde{C}_t &= \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \\ C_t &= F_t \odot C_{t-1} + I_t \odot \tilde{C}_t \\ H_t &= O_t \odot \tanh(C_t) \end{aligned}$$



多了个c，c的数值区间比较大，记忆单元，要实现的效果和gru一样，要么是多去看看现在的xt，要么是多去看看过去的状态

6、深度循环神经网络

更深



$$\begin{aligned} H_t^1 &= f_1(H_{t-1}^1, X_t) \\ &\dots \\ H_t^j &= f_j(H_{t-1}^j, H_t^{j-1}) \\ &\dots \\ O_t &= g(H_t^L) \end{aligned}$$

第一个隐藏层 ht 是根据 xt 算过来的

第 j 层用的是上一个隐藏层的状态

总结：深度循环神经网络使用多个隐藏层来获得更多的非线性性

7、双向循环神经网络

完形填空

——双向 lstm 非常不适合用来做推理，主要是对一个句子做特征提取（翻译、改写）

推理

• 训练：



• 推理：



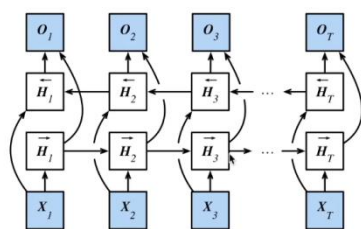
未来很重要

I am _____
I am _____ very hungry,
I am _____ very hungry, I could eat half a pig.

- 取决于过去和未来的上下文，可以填很不一样的
- 目前为止RNN只看过去
- 在填空的时候，我们也可以看未来



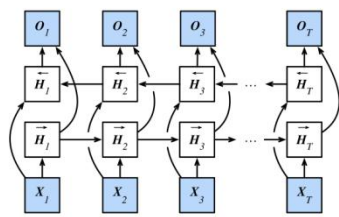
双向 RNN



- 一个前向RNN隐层
- 一个反向RNN隐层
- 合并两个隐状态得到输出



双向 RNN



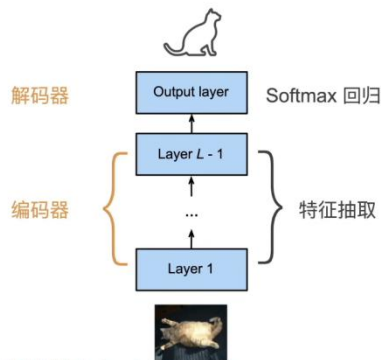
$$\begin{aligned}\vec{H}_t &= \phi(\mathbf{X}_t \mathbf{W}_{xh}^{(f)} + \vec{H}_{t-1} \mathbf{W}_{hh}^{(f)} + \mathbf{b}_h^{(f)}), \\ \leftarrow H_t &= \phi(\mathbf{X}_t \mathbf{W}_{xh}^{(b)} + \leftarrow H_{t+1} \mathbf{W}_{hh}^{(b)} + \mathbf{b}_h^{(b)}), \\ \mathbf{H}_t &= [\vec{H}_t, \leftarrow H_t] \\ \mathbf{O}_t &= \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q\end{aligned}$$

总结

- 双向循环神经网络通过反向更新的隐藏层来利用方向时间信息
- 通常用来对序列抽取特征、填空，而不是预测未来

8、编码器 encode-解码器 decode

重新考察CNN

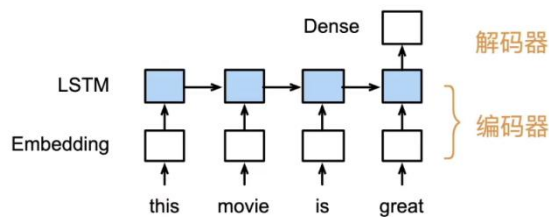


- 编码器：将输入编程成中间表达形式（特征）
- 解码器：将中间表示解码成输出



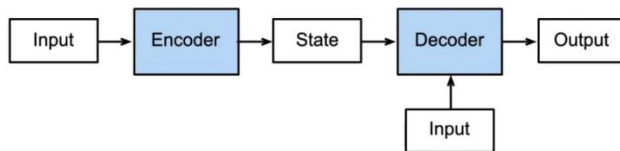
重新考察RNN

- 编码器：将文本表示成向量
- 解码器：向量表示成输出



编码器-解码器架构

- 一个模型被分为两块：
 - 编码器处理输入
 - 解码器生成输出



一个模型被分成两块：encoder、decoder

——encoder：拿到一个输入，把这个东西表示成中间的状态

——decoder：拿到这个东西之后，decoder 把它变成输出，decoder 也可以有额外的输入

总结

- 使用编码器-解码器架构的模型，编码器负责表示输入，解码器负责输出

9、序列到序列学习 (seq2seq)

把一个句子翻译到另一个句子 (dna 转 rna)

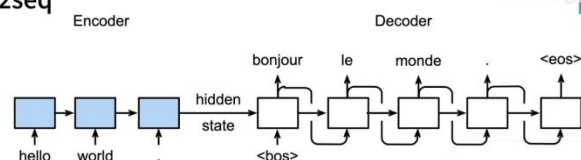
机器翻译

- 给定一个源语言的句子，自动翻译成目标语言
- 这两个句子可以有不同的长度



Seq2seq 是 encoder-decoder 架构

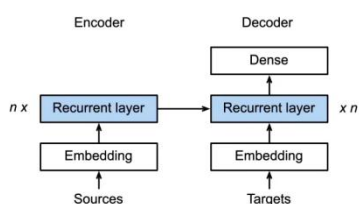
Seq2seq



- 编码器是一个RNN，读取输入句子
 - 可以是双向
- 解码器使用另外一个RNN来输出



编码器-解码器细节

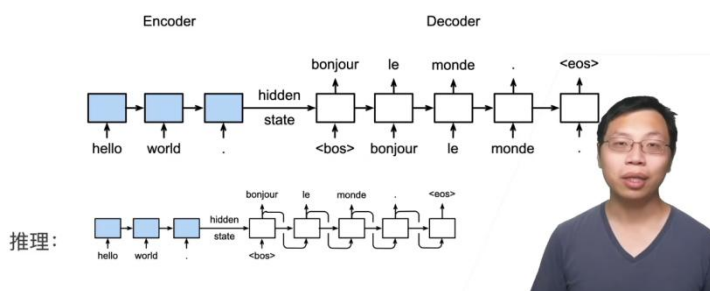


- 编码器是没有输出的RNN
- 编码器最后时间步的隐状态用作解码器的初始隐状态



训练

- 训练时解码器使用目标句子作为输入



衡量生成序列的好坏的BLEU

- p_n 是预测中所有 n-gram 的精度
 - 标签序列 $A B C D E F$ 和预测序列 $A B B C D$, 有 $p_1 = 4/5$, $p_2 = 3/4$, $p_3 = 1/3$, $p_4 = 0$
- BLEU定义

$$\exp \left(\min \left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}} \right) \right) \prod_{n=1}^k p_n^{1/2^n}$$

惩罚过短的预测

长匹配有高权重

N-gram 的精度: p_1 、 p_2 里的 1、2 指的是序列长度, p 指的是序列长度预测准确的概率

防止预测过短, 因为越短越会命中精度

$P \leq 1$, ———— 长匹配有高权重

总结

- Seq2seq 从一个句子生成另一个句子
- 编码器和解码器都是 RNN
- 将编码器最后时间隐状态来初始解码器隐状态来完成信息传递

- 常用 BLEU 来衡量生成序列的好坏

10、束搜索

贪心搜索

- 在 seq2seq 中我们使用了贪心搜索来预测序列
- 将当前时刻预测概率最大的词输出
- 但贪心很可能不是最优的：

贪心: $0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$ 很好的选项: $0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$

Time step	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

Time step	1	2	3	4
A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.1	0.6

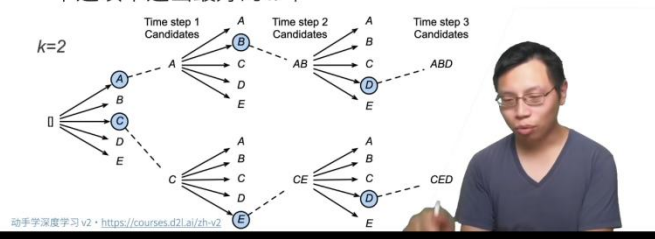
穷举搜索

- 最优算法：对所有可能的序列，计算它的概率，然后选取最好的那个
- 如果输出字典大小为 n ，序列最长为 T ，那么我们需要考察 n^T 个序列
- $n = 10000$, $T = 10$: $n^T = 10^{40}$
- 计算上不可行

取个贪心和搜索中间的东西

束搜索

- 保存最好的 k 个候选
- 在每个时刻，对每个候选新加一项 (n 种可能)，在 kn 个选项中选出最好的 k 个



束搜索

- 时间复杂度 $O(knT)$
- $k = 5$, $n = 10000$, $T = 10$: $knT = 5 \times 10^5$
- 每个候选的最终分数是：

$$\frac{1}{L^\alpha} \log p(y_1, \dots, y_L) = \frac{1}{L^\alpha} \sum_{t'=1}^L \log p(y_{t'} | y_1, \dots, y_{t'-1})$$

- 通常 $\alpha = 0.75$

总结

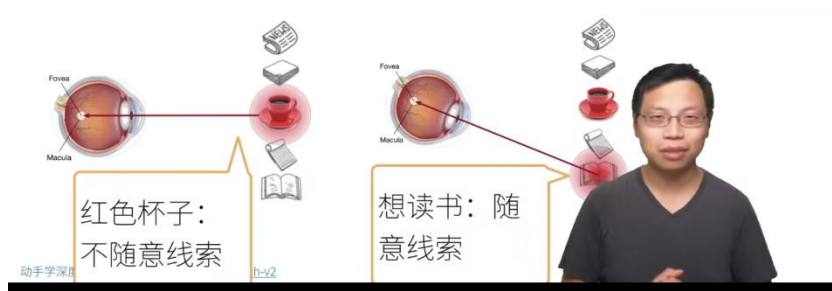
- 束搜索在每次搜索时保存 k 个最好的候选
 - $k = 1$ 时是贪心搜索
 - $k = n$ 时是穷举搜索

五、注意力机制

1、注意力机制

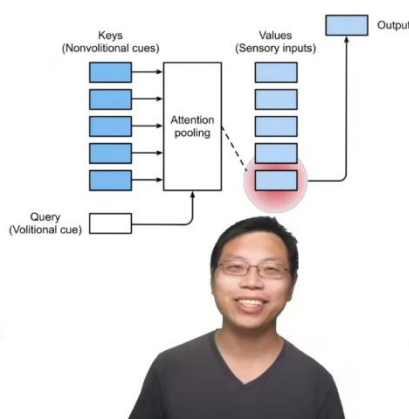
心理学

- 动物需要在复杂环境下有效关注值得注意的点
- 心理学框架：人类根据随意线索和不随意线索选择注意点



注意力机制

- 卷积、全连接、池化层都只考虑不随意线索
- 注意力机制则显示的考虑随意线索
 - 随意线索被称之为查询（query）
 - 每个输入是一个值（value）和不随意线索（key）的对
 - 通过注意力池化层来有偏向性的选择选择某些输入



注意力机制——显式建模随意线索

——根据 query 去有偏向的选择某些 key value pair

——区别于之前将过的所有层，显式地加了 query，会根据查询去寻找比较感兴趣的东西

- 和池化层不一样，多了 query
- 区别于别的模型的地方

非参注意力池化层

- 给定数据 $(x_i, y_i), i = 1, \dots, n$
- 平均池化是最简单的方案: $f(x) = \frac{1}{n} \sum_i y_i$
- 更好的方案是60年代提出来的Nadaraya-Watson核回归

$$f(x) = \sum_{i=1}^n \frac{K(x - x_i)}{\sum_{j=1}^n K(x - x_j)} y_i$$

query key value



- x 是 query，要查询的东西，xj 是所有的 keyvalue，候选的东西
- K——衡量 x 和 xi 之间距离的函数
- 加权对 yi 求和
- 找相近的数据
- 非参的注意力池化层

Nadaraya-Watson核回归

- 使用高斯核 $K(u) = \frac{1}{\sqrt{2\pi}} \exp(-\frac{u^2}{2})$

- 那么

$$\begin{aligned} f(x) &= \sum_{i=1}^n \frac{\exp\left(-\frac{1}{2}(x - x_i)^2\right)}{\sum_{j=1}^n \exp\left(-\frac{1}{2}(x - x_j)^2\right)} y_i \\ &= \sum_{i=1}^n \text{softmax}\left(-\frac{1}{2}(x - x_i)^2\right) y_i \end{aligned}$$

exp 好处——match 到大于 0 的东西

参数化的注意力机制

- 在之前基础上引入可以学习的 w

$$f(x) = \sum_{i=1}^n \text{softmax} \left(-\frac{1}{2}((x - x_i)w)^2 \right) y_i$$

总结

- 心理学认为人通过随意线索和不随意线索选择注意点
- 注意力机制中，通过query（随意线索）和key（不随意线索）来有偏向性的选择输入
 - 可以一般的写作 $f(x) = \sum_i \alpha(x, x_i) y_i$ ，这里 $\alpha(x, x_i)$ 是注意力权重
- 早在60年代就有非参数的注意力机制
- 接下来我们会介绍多个不同的权重设计



2、注意力分数

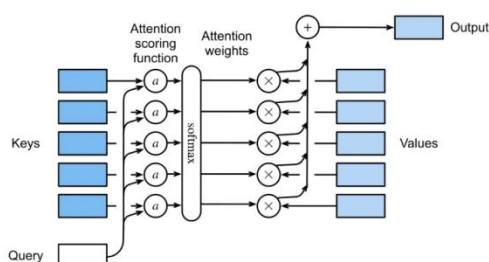
——注意力权重的设计

注意力分数

注意力权重

注意力分数

- 回顾： $f(x) = \sum_i \alpha(x, x_i) y_i = \sum_{i=1}^n \text{softmax} \left(-\frac{1}{2}(x - x_i)^2 \right) y_i$



给我一个 query，所有结果等价于所有的 key value pair 跟他算一个距离
——权重大于等于 0，加起来等于 1；分数没有 normalize
——注意力分数给到 softmax 变成注意力权重，加权和得到最终输出

拓展到高维度

- 假设query $\mathbf{q} \in \mathbb{R}^q$, m 对 key-value $(\mathbf{k}_1, \mathbf{v}_1), \dots$, 这里 $\mathbf{k}_i \in \mathbb{R}^k, \mathbf{v}_i \in \mathbb{R}^v$
- 注意力池化层:

$$f(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i \in \mathbb{R}^v$$
$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))} \in \mathbb{R}$$

注意力分数



怎么把 key-value 拓展到高维度

query 是长为 q 的向量, 有 m 对 key-value 的值,

——对每个 q 跟每个 key 做一个注意力的权重, 再乘以 $\mathbf{v}_i$, 全部加起来, 最后输出等于一个 v 的东西

α 设计的两个思路:

Additive Attention

- 可学参数: $\mathbf{W}_k \in \mathbb{R}^{h \times k}, \mathbf{W}_q \in \mathbb{R}^{h \times q}, \mathbf{v} \in \mathbb{R}^h$

$$a(\mathbf{k}, \mathbf{q}) = \mathbf{v}^T \tanh(\mathbf{W}_k \mathbf{k} + \mathbf{W}_q \mathbf{q})$$

- 等价于将key和value合并起来后放入到一个隐藏大小为 h 输出大小为 1 的单隐藏层MLP



——加性注意力, query 是一个长为 q 的东西, key 是一个长为 k 的东西, 这两个东西并在一起, 变成了一个长为 $k+q$ 的向量, 然后丢到一个隐藏层大小为 h , 输出大小为 1 的但隐藏层 mlp

——好处: key、value、query 可以任意长度

如果 q 和 k_i 都是长为 d 的向量

Scaled Dot-Product Attention

- 如果query和key都是同样的长度 $\mathbf{q}, \mathbf{k}_i \in \mathbb{R}^d$, 那么可以

$$a(\mathbf{q}, \mathbf{k}_i) = \langle \mathbf{q}, \mathbf{k}_i \rangle / \sqrt{d}$$

- 向量化版本

- $\mathbf{Q} \in \mathbb{R}^{n \times d}, \mathbf{K} \in \mathbb{R}^{m \times d}, \mathbf{V} \in \mathbb{R}^{m \times v}$
- 注意力分数: $a(\mathbf{Q}, \mathbf{K}) = \mathbf{Q}\mathbf{K}^T / \sqrt{d} \in \mathbb{R}^{n \times m}$
- 注意力池化: $f = \text{softmax}(a(\mathbf{Q}, \mathbf{K})) \mathbf{V} \in \mathbb{R}^{n \times v}$



——对每一行做 softmax

总结

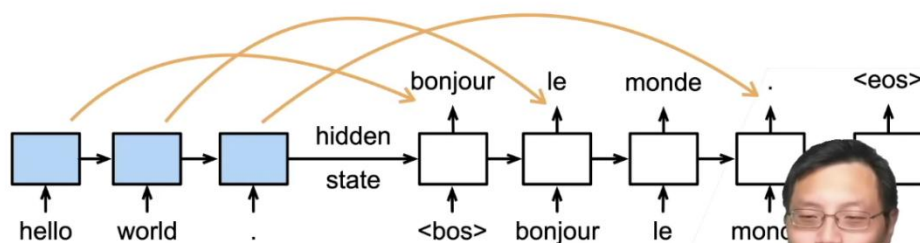
- 注意力分数是query和key的相似度，注意力权重是分数的 softmax 结果
- 两种常见的分数计算：
 - 将query和key合并起来进入一个单输出单隐藏层的MLP
 - 直接将query和key做内积

——叫相似度是因为没有被 normalize 过

3、使用注意力机制的 seq2seq

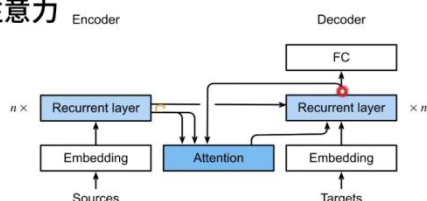
动机

- 机器翻译中，每个生成的词可能相关于源句子中不同的词



- seq2seq模型中不能对此直接建模

加入注意力



- 编码器对每次词的输出作为key和value（它们相关的）
- 解码器RNN对上一个词的输出是query
- 注意力的输出和下一个词的词嵌入合并进入

总结

- Seq2seq中通过隐状态在编码器和解码器中传递信息
- 注意力机制可以根据解码器RNN的输出来匹配到合适的编码器RNN的输出来更有效的传递信息

4、自注意力和位置编码

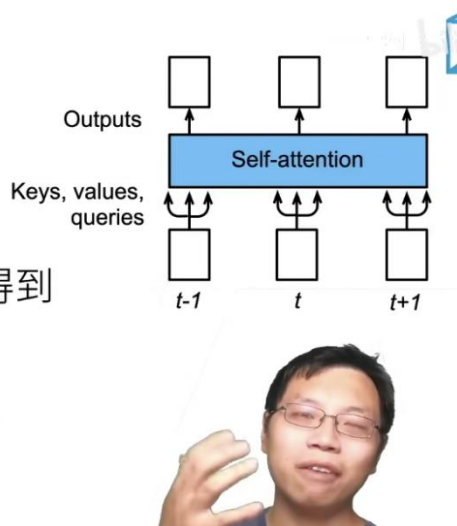
Self-attention

——自注意力的 key、value、query 怎么选

自注意力

- 给定序列 $\mathbf{x}_1, \dots, \mathbf{x}_n, \forall \mathbf{x}_i \in \mathbb{R}^d$
- 自注意力池化层将 $\mathbf{x}_i$ 当做key, value, query来对序列抽取特征得到 $\mathbf{y}_1, \dots, \mathbf{y}_n$, 这里

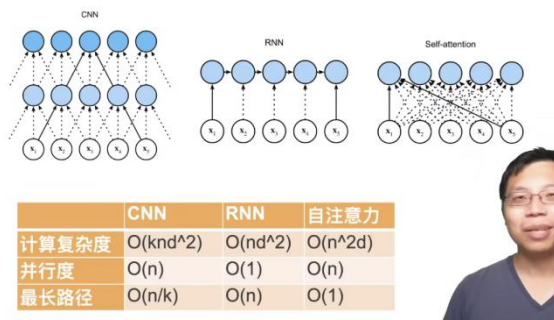
$$\mathbf{y}_i = f(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n)) \in \mathbb{R}^d$$



- 给定一个序列 x_1 到 x_n , 自注意力将 x_i 又当 key 又当 value、又当 query
- 给定序列放进去之后就可以对每个元素进行输出

Cnn、rnn 和自注意力都可以处理序列

跟CNN，RNN对比



rnn 并行度很差，算完前面才能到后面
自注意力并行度好

——自注意力比较适合处理比较长的文本，它的设计使得它可以无视距离（每次能看所有，能够并行）

——计算复杂度高，transformer、bert、gpt 计算量特别大

位置编码

位置编码

- 跟CNN/RNN不同，自注意力并没有记录位置信息
- 位置编码将位置信息注入到输入里
 - 假设长度为 n 的序列是 $\mathbf{X} \in \mathbb{R}^{n \times d}$ ，那么使用位置编码矩阵 $\mathbf{P} \in \mathbb{R}^{n \times d}$ 来输出 $\mathbf{X} + \mathbf{P}$ 作为自编码输入
- $\mathbf{P}$ 的元素如下计算：

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$



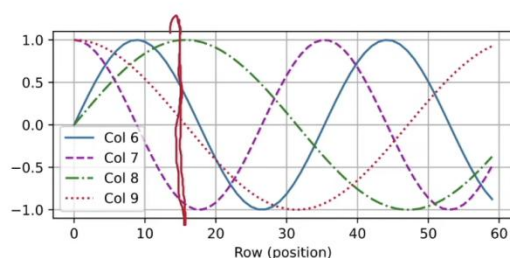
——纯用自注意力机制做序列模型，没有位置信息有问题

——位置编码不是把位置信息加到模型里面，不改变每次能看所有，能够并行，把位置编码信息放到输入里面，让输入有位置信息

——奇数列是 $\cos$ 函数，偶数列是 $\sin$ 函数，不同列的周期不一样

位置编码矩阵

$$\mathbf{P} \in \mathbb{R}^{n \times d}: p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$

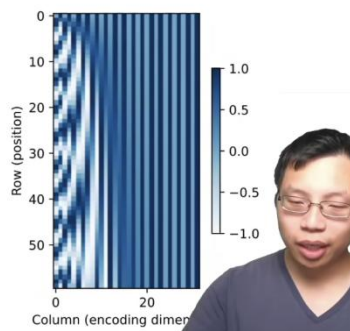


绝对位置信息

计算机使用的二进制编码

```
0 in binary is 000
1 in binary is 001
2 in binary is 010
3 in binary is 011
4 in binary is 100
5 in binary is 101
6 in binary is 110
7 in binary is 111
```

位置编码矩阵



相对位置信息

- 位于 $i+\delta$ 处的位置编码可以线性投影位置 i 处的位置编码来表示
- 记 $\omega_j = 1/10000^{2j/d}$, 那么

$$\begin{bmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} = \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix}$$

投影矩阵
跟 i 无关

动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>



总结

- 自注意力池化层将 $\mathbf{x}_i$ 当做key, value, query来对序列抽取特征
- 完全并行、最长序列为1、但对长序列计算复杂度高
- 位置编码在输入中加入位置信息, 使得自注意力能够记忆位置信息

5、Transformer

Transformer 架构

- 基于编码器-解码器架构来处理序列对
- 跟使用注意力的seq2seq不同，Transformer是纯基于注意力

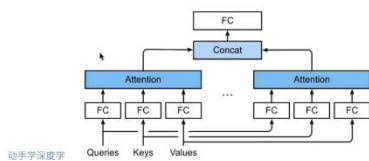


动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>

——和 encoder、decoder 架构、seq2seq 架构没有本质区别，有 n 层，每一层叫 transformer 块，信息传递最后一层的输出作为之后层的输入完成信息的传递

多头注意力

- 对同一 key, value, query, 希望抽取不同的信息
 - 例如短距离关系和长距离关系
- 多头注意力使用 h 个独立的注意力池化
 - 合并各个头 (head) 输出得到最终输出



多头注意力

- query $\mathbf{q} \in \mathbb{R}^{d_q}$, key $\mathbf{k} \in \mathbb{R}^{d_k}$, value $\mathbf{v} \in \mathbb{R}^{d_v}$
- 头 i 的可学习参数 $\mathbf{W}_i^{(q)} \in \mathbb{R}^{p_q \times d_q}$, $\mathbf{W}_i^{(k)} \in \mathbb{R}^{p_k \times d_k}$, $\mathbf{W}_i^{(v)} \in \mathbb{R}^{p_v \times d_v}$
- 头 i 的输出 $\mathbf{h}_i = f(\mathbf{W}_i^{(q)}\mathbf{q}, \mathbf{W}_i^{(k)}\mathbf{k}, \mathbf{W}_i^{(v)}\mathbf{v}) \in \mathbb{R}^{p_v}$
- 输出的可学习参数 $\mathbf{W}_o \in \mathbb{R}^{p_o \times hp_v}$
- 多头注意力的输出

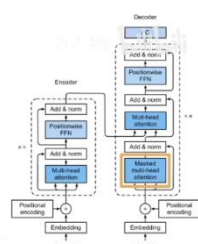
$$\mathbf{W}_o \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_h \end{bmatrix} \in \mathbb{R}^{p_o}$$

动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>



有掩码的多头注意力

- 解码器对序列中一个元素输出时，不应该考虑该元素之后的元素
- 可以通过掩码来实现
 - 也就是计算 x_i 输出时，假装当前序列长度为 i

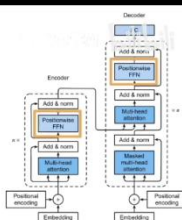


——attention 没有时间信息，在中间第 i 个输出的时候可以看到后面所有东西
编码没关系，解码不行

——通过掩码来实现

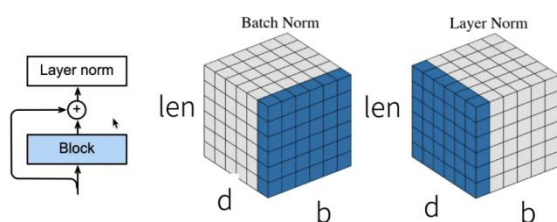
基于位置的前馈网络

- 将输入形状由 (b, n, d) 变换成 (bn, d)
- 作用两个全连接层
- 输出形状由 (bn, d) 变化回 (b, n, d)
- 等价于两层核窗口为1的一维卷积层



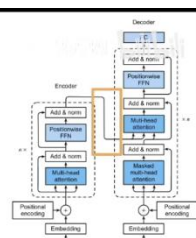
层归一化

- 批量归一化对每个特征/通道里元素进行归一化
 - 不适合序列长度会变的NLP应用
- 层归一化对每个样本里的元素进行归一化



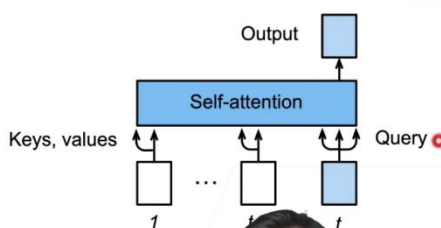
信息传递

- 编码器中的输出 $y_1, \dots, y_n$
- 将其作为解码中第 i 个Transformer块中多头注意力的key和value
 - 它的query来自目标序列
- 意味着编码器和解码器中块的个数和输出维度都是一样的



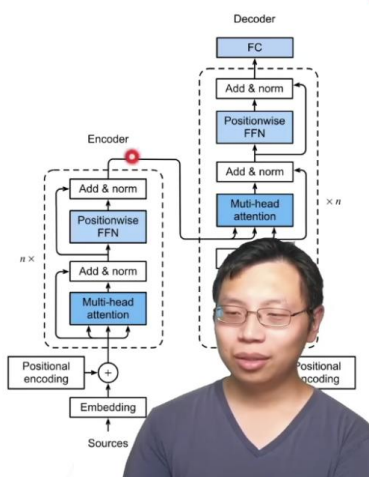
预测

- 预测第 $t+1$ 个输出时
- 解码器中输入前 t 个预测值
 - 在自注意力中，前 t 个预测值作为key和value，第 t 个预测值还作为query



总结

- Transformer是一个纯使用注意力的编码-解码器
- 编码器和解码器都有 n 个 transformer 块
- 每个块里使用多头（自）注意力，基于位置的前馈网络，和层归一化



6、BERT

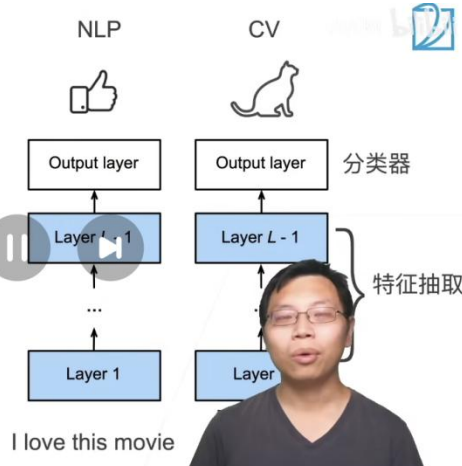
——针对 nlp 的微调设计

NLP里的迁移学习

- 使用预训练好的模型来抽取词、句子的特征
 - 例如 word2vec 或 语言模型
- 不更新预训练好的模型
- 需要构建新的网络来抓取新任务需要的信息
 - Word2vec 忽略了时序信息，
语言模型只看了一个方向

BERT的动机

- 基于微调的NLP模型
- 预训练的模型抽取了足够的信息
- 新的任务只需要增加一个简单的输出层



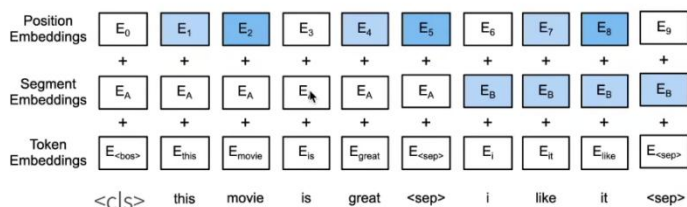
BERT 架构

- 只有编码器的 Transformer
- 两个版本：
 - Base: #blocks = 12, hidden size = 768, #heads = 12, #parameters = 110M
 - Large: #blocks = 24, hidden size = 1024, #heads = 16, #parameter = 340M
- 在大规模数据上训练 > 3B 词



对输入的修改

- 每个样本是一个句子对
- 加入额外的片段嵌入
- 位置编码可学习



预训练任务1：带掩码的语言模型

- Transformer的编码器是双向，标准语言模型要求单向
- 带掩码的语言模型每次随机（15%概率）将一些词元换成 <mask>
- 因为微调任务中不出现 <mask>
 - 80%概率下，将选中的词元变成 <mask>
 - 10%概率下换成一个随机词元
 - 10%概率下保持原有的词元



预训练任务2：下一句子预测

- 预测一个句子对中两个句子是不是相邻
- 训练样本中：
 - 50%概率选择相邻句子对： <cls> this movie is great <sep> i like it <sep>
 - 50%概率选择随机句子对： <cls> this movie is great <sep> hello world <sep>
- 将 <cls> 对应的输出放到一个全连接层来预测



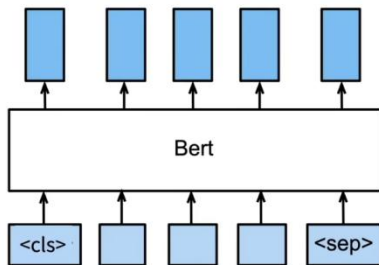
总结

- BERT针对微调设计
- 基于Transformer的编码器做了如下修改
 - 模型更大，训练数据更多
 - 输入句子对，片段嵌入，可学习的位置编码
 - 训练时使用两个任务：
 - 带掩码的语言模型
 - 下一个句子预测

很少做完形填空和预测两个句子是否同时出现
bert 主要用来做微调

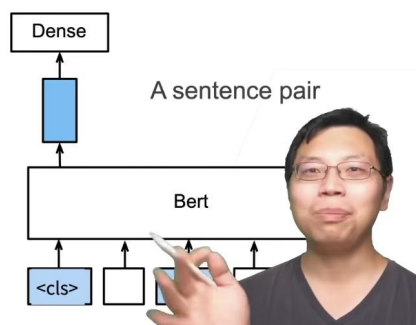
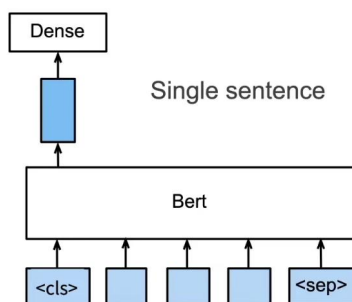
微调 Bert

- BERT 对每一个词元返回抽取了上下文信息的特征向量
- 不同的任务使用不同的特性



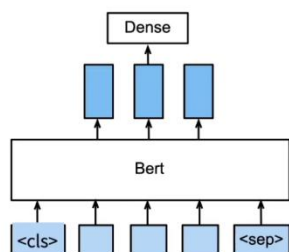
句子分类

- 将 <cls> 对应的向量输入到全连接层分类



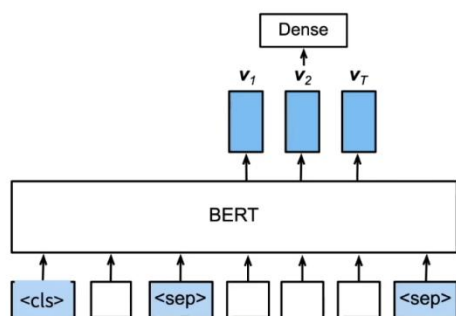
命名实体识别

- 识别一个词元是不是命名实体，例如人名、机构、位置
- 将非特殊词元放进全连接层分类



问题回答

- 给定一个问题，和描述文字，找出一个片段作为回答
- 对片段中的每个词元预测它是不是回答的开头或结束



动手学深度学习 v2 · <https://courses.d2l.ai/zh-v2>



总结

- 即使下游任务各有不同，使用BERT微调时均只需要增加输出层
- 但根据任务的不同，输入的表达，和使用的BERT特征也会不一样



7、优化算法

优化问题

- 一般形式

$$\text{minimize } f(\mathbf{x}) \quad \text{subject to } \mathbf{x} \in C$$

- 目标函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$

- 限制集合例子

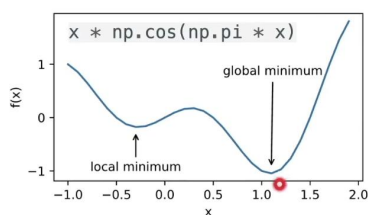
$$C = \{\mathbf{x} \mid h_1(\mathbf{x}) = 0, \dots, h_m(\mathbf{x}) = 0, g_1(\mathbf{x}) \leq 0, \dots, g_r(\mathbf{x}) \leq 0\}$$

- 如果 $C = \mathbb{R}^n$ 那就是不受限



局部最小 vs 全局最小

- 全局最小 $\mathbf{x}^*$: $f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \in C$
- 局部最小 $\mathbf{x}^*$: 存在 ε , 使得 $f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} : \|\mathbf{x} - \mathbf{x}^*\| \leq \varepsilon$
- 使用迭代优化算法来求解, 一般只能保证找到局部最小值



动手学深度学习 v2 • <https://courses.d2l.ai/zh-v2>

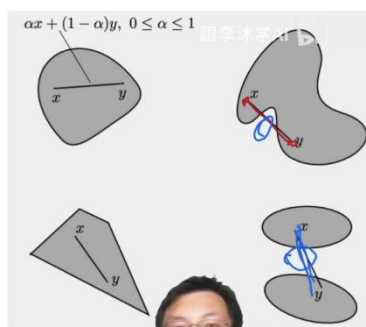


凸集

- 一个 $\mathbb{R}^n$ 的子集 C 是凸当且仅当

$$\alpha \mathbf{x} + (1 - \alpha) \mathbf{y} \in C$$

$$\forall \alpha \in [0, 1] \quad \forall \mathbf{x}, \mathbf{y} \in C$$



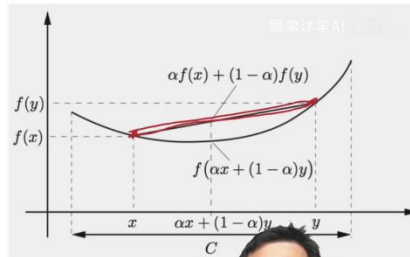
在里面找任意两个点, 连成线, 线都在这个区域里面

凸函数

- 函数 $f: C \rightarrow \mathbb{R}$ 是凸当且仅当

$$\begin{aligned} f(\alpha \mathbf{x} + (1 - \alpha)\mathbf{y}) \\ \leq \alpha f(\mathbf{x}) + (1 - \alpha)f(\mathbf{y}) \\ \forall \alpha \in [0, 1] \quad \forall \mathbf{x}, \mathbf{y} \in C \end{aligned}$$

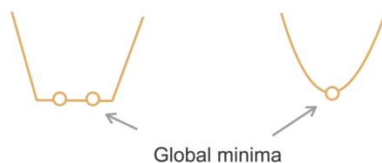
- 如果 $\mathbf{x} \neq \mathbf{y}, \alpha \in (0, 1)$ 时不等式严格成立, 那么叫严格凸函数



凸函数：两个点连成线，保证说整个函数都在连线的下面

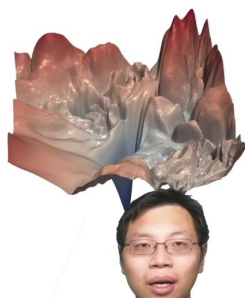
凸函数优化

- 如果代价函数 f 是凸的，且限制集合 C 是凸的，那么就是凸优化问题，那么局部最小一定是全局最小
- 严格凸优化问题有唯一的全局最小



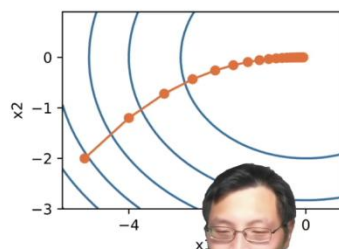
凸和非凸例子

- 凸
 - 线性回归 $f(\mathbf{x}) = \|\mathbf{W}\mathbf{x} - \mathbf{b}\|_2^2$
 - Softmax 回归
- 非凸：其他
 - MLP, CNN, RNN, attention, ...



梯度下降

- 最简单的迭代求解算法
- 选取开始点 $\mathbf{x}_0$
- 对 $t = 1, \dots, T$
 - $\mathbf{x}_t = \mathbf{x}_{t-1} - \eta \nabla f(\mathbf{x}_{t-1})$
- η 叫做学习率



随机梯度下降

- 有 n 个样本时，计算

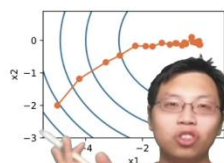
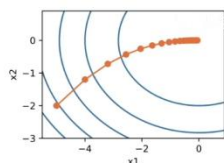
$$f(\mathbf{x}) = \frac{1}{n} \sum_{i=0}^n \ell_i(\mathbf{x}) \text{ 的导数太贵}$$

- 随机梯度下降在时间 t 随机选项样本 t_i 来近似 $f(\mathbf{x})$

$$\mathbf{x}_t = \mathbf{x}_{t-1} - \eta_t \nabla \ell_{t_i}(\mathbf{x}_{t-1})$$

$$\mathbb{E}[\nabla \ell_{t_i}(\mathbf{x})] = \mathbb{E}[\nabla f(\mathbf{x})]$$

——因为导数是线性可加的



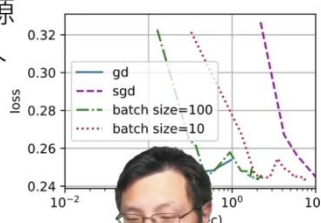
小批量随机梯度下降

- 计算单样本的梯度难完全利用硬件资源
- 小批量随机梯度下降在时间 t 采样一个随机子集 $I_t \subset \{1, \dots, n\}$ 使得 $|I_t| = b$

$$\mathbf{x}_t = \mathbf{x}_{t-1} - \frac{\eta_t}{b} \sum_{i \in I_t} \nabla \ell_i(\mathbf{x}_{t-1})$$

- 同样，这是一个无偏的近似，但降低了方差

$$\mathbb{E}\left[\frac{1}{b} \sum_{i \in I_t} \nabla \ell_i(\mathbf{x})\right] = \nabla f(\mathbf{x})$$



冲量法

- 冲量法使用平滑过的梯度对权重更新

$$\mathbf{g}_t = \frac{1}{b} \sum_{i \in I_t} \nabla \ell_i(\mathbf{x}_{t-1})$$

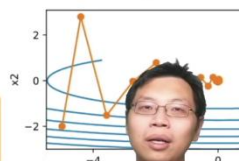
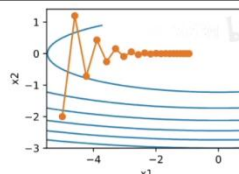
$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t \quad \mathbf{w}_t = \mathbf{w}_{t-1} - \eta \mathbf{v}_t$$

梯度平滑: $\mathbf{v}_t = \mathbf{g}_t + \beta \mathbf{g}_{t-1} + \beta^2 \mathbf{g}_{t-2} + \beta^3 \mathbf{g}_{t-3} + \dots$

- β 常见取值 $[0.5, 0.9, 0.95, 0.99]$

维护一个惯性

——看过去时间的梯度



Adam

- 记录 $\mathbf{v}_t = \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t$ 通常 $\beta_1 = 0.9$
- 展开 $\mathbf{v}_t = (1 - \beta_1)(\mathbf{g}_t + \beta_1 \mathbf{g}_{t-1} + \beta_1^2 \mathbf{g}_{t-2} + \beta_1^3 \mathbf{g}_{t-3} + \dots)$
- 因为 $\sum_{i=0}^{\infty} \beta_1^i = \frac{1}{1 - \beta_1}$, 所以权重和为1
- 由于 $\mathbf{v}_0 = 0$, 且 $\sum_{i=0}^t \beta_1^i = \frac{1 - \beta_1^{t+1}}{1 - \beta_1}$,
修正 $\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_1^t}$



- 对学习率没那么敏感，非常平滑
- 用的非常多
- $\mathbf{v}_t$ 是过去所有东西权重的加权和
- 当 t 比较小的时候，修正 $\mathbf{v}_t$

Adam

- 类似记录 $\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$, 通常 $\beta_2 = 0.999$, 且修正
 $\hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t}$
- 计算重新调整后的梯度 $\mathbf{g}'_t = \frac{\hat{\mathbf{v}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}$
- 最后更新 $\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \mathbf{g}'_t$



总结

- 深度学习模型大多是非凸
- 小批量随机梯度下降是最常用的优化算法
- 冲量对梯度做平滑
- Adam对梯度做平滑，且对梯度各个维度值做重新调整

